

Resumen

Este proyecto tiene como objetivo el diseño e implementación de un sistema inteligente integrado en una plataforma digital de comparación de precios de carburantes, desarrollada sobre Google Cloud Platform con una arquitectura de microservicios desplegada mediante Cloud Run y Cloud Functions. La solución, implementada con FastAPI en el backend y React en el frontend, permite a los usuarios consultar precios actualizados en tiempo real a partir de la API oficial del Ministerio para la Transición Ecológica, acceder a históricos de precios por estación de servicio y visualizar predicciones a siete días generadas mediante modelos de machine learning basados en LightGBM. Más allá de la simple visualización de información, el sistema incorpora un módulo de recomendación de repostaje en ruta que, apoyándose en la API de Google Maps para el análisis geoespacial y la optimización de recorridos, determina qué estación resulta más conveniente económicamente para un trayecto definido por el usuario. Esta recomendación combina el precio actual, la evolución prevista y la desviación necesaria respecto al recorrido, evaluando el coste total del desplazamiento. Los datos son gestionados mediante Neon Tech como almacén analítico principal y Cloud Storage para la persistencia de modelos y artefactos. La plataforma es accesible desde dispositivos móviles mediante una interfaz web responsive y contempla adicionalmente un asistente conversacional, priorizando en todo momento la experiencia de usuario. El proyecto integra competencias de ingeniería de software, análisis de datos e inteligencia artificial aplicada, constituyendo una solución tecnológica orientada al ahorro económico, la eficiencia energética y la toma de decisiones basada en datos.

Descriptores

Computación en la nube, predicción de precios, plataforma de carburantes, recomendación geoespacial.

Índice general

1	Capítulo Primero - Introducción	1
1.1	Motivación	1
1.2	Estructura de la memoria	2
2	Capítulo Segundo - Antecedentes y Justificación	3
2.1	Situación actual del sector de gasolineras	3
2.2	Estado del arte	3
2.2.1	Predicción de precios de combustibles	3
2.2.2	Sistemas de recomendación en movilidad	4
2.2.3	Arquitecturas <i>cloud</i> para datos en tiempo real	4
2.2.4	Asistentes conversacionales en aplicaciones de dominio específico	5
2.3	Soluciones existentes y limitaciones	6
2.3.1	Plataformas gubernamentales	6
2.3.2	Aplicaciones de terceros	6
2.3.3	Síntesis comparativa	6
2.4	Origen del proyecto y evolución	7
2.5	Justificación técnica y académica	8
2.5.1	Alineación con los Objetivos de Desarrollo Sostenible	8
3	Capítulo tercero - Marco tecnológico	9
3.1	Capa de presentación: React y Leaflet.js	9
3.2	Capa de gateway: Hono sobre Node.js	9
3.3	Capa de servicios de negocio: FastAPI y Fastify	10
3.4	Capa de datos: PostgreSQL y Apache Parquet	10
3.5	Capa de inteligencia: LightGBM	10
3.6	Capa de enrutamiento geoespacial: OpenRouteService	11

3.7	Capa de cartografía: OpenStreetMap y Nominatim	11
3.8	Capa de inteligencia conversacional: Gemini y Google GenAI	11
3.9	Plataforma de despliegue: Google Cloud	11
3.10	Síntesis: justificación del stack tecnológico	12
4	Capítulo Cuarto - Objetivos y Alcance	13
4.1	Objetivo general	13
4.2	Objetivos específicos y medibles	13
4.2.1	Ingesta y gestión de datos	13
4.2.2	Consulta y visualización geoespacial	14
4.2.3	Predicción de precios	14
4.2.4	Recomendación basada en rutas	14
4.2.5	Arquitectura, despliegue y calidad	14
4.3	Alcance del sistema	15
4.3.1	Ámbito geográfico y de datos	15
4.3.2	Componentes funcionales incluidos	15
4.3.3	Entorno de despliegue	16
4.4	Limitaciones	17
4.5	Exclusiones	17
5	Capítulo Quinto - Planificación del Proyecto	19
5.1	Descomposición en tareas (EDT)	19
5.2	Estimación de tiempos	20
5.3	Cronograma (Diagrama de Gantt)	21
5.4	Hitos del proyecto	22
5.5	Encadenamiento de actividades	22
5.6	Plan de recursos humanos	23
5.7	Replanificaciones realizadas	24
6	Capítulo Sexto - Presupuesto y evaluación de costes	26
6.1	Costes laborales	26
6.2	Material, software e infraestructura	27
6.3	Coste total estimado del proyecto	27
6.4	Estimación del coste en producción	28
7	Capítulo Séptimo	30

8	Capítulo Octavo - Desarrollo del Sistema	31
8.1	Especificación de Requisitos	31
8.1.1	Contexto del sistema	31
8.1.2	Identificación de usuarios	31
8.1.3	Requisitos funcionales	32
8.1.4	Requisitos no funcionales	32
8.1.5	Restricciones	33
8.2	Diseño del Sistema	33
8.2.1	Alternativas exploradas y decisiones tecnológicas	33
8.2.2	Arquitectura general	34
8.2.3	Diseño lógico y físico	34
8.2.4	Modelo de datos	35
8.2.5	Diagramas UML	36
8.3	Consideraciones sobre la Implementación	37
8.3.1	Estructura del proyecto	37
8.3.2	Servicio de gasolineras y sincronización de datos	38
8.3.3	Servicio de usuarios y autenticación	39
8.3.4	Servicio de recomendación de ruta	40
8.3.5	Módulo de predicción de precios	40
8.3.6	Asistente conversacional de voz	42
8.3.7	Frontend: React PWA con Nginx	43
8.3.8	Despliegue en GCP y pipeline CI/CD	44
8.3.9	Seguridad transversal	45
8.4	Plan de Pruebas	47
8.4.1	Estrategia	47
8.4.2	Pruebas unitarias	47
8.4.3	Pruebas de integración	47
8.4.4	Pruebas funcionales	47
8.4.5	Resultados obtenidos	47
8.5	Manual de Usuario	47
8.5.1	Acceso al sistema	47
8.5.2	Uso de la plataforma	47
8.5.3	Capturas de pantalla	47
8.6	Incidencias del Proyecto	47

Bibliografía

48

1. CAPÍTULO PRIMERO - INTRODUCCIÓN

Esta memoria recoge el trabajo realizado en el marco del Proyecto de Fin de Grado del Grado en Ingeniería Informática. Este proyecto integra de forma aplicada los conocimientos y competencias adquiridas a lo largo de la titulación, especialmente en las áreas de ingeniería del software, arquitecturas en la nube, gestión y procesamiento de datos, sistemas inteligentes y análisis predictivo.

El proyecto desarrollado consiste en la evolución del prototipo académico *TankGo* [1] hacia una plataforma SaaS [2] orientada a la consulta, comparación y análisis de precios de combustibles y puntos de recarga eléctrica. La solución propuesta se concibe como un sistema integrado desplegado en entorno cloud que combina distintos componentes tecnológicos: ingestión y persistencia de datos, procesamiento distribuido, modelos de predicción basados en aprendizaje automático, algoritmos de recomendación y un sistema de asistencia conversacional apoyado en los emergentes LLMs para la interacción con el usuario.

Este proyecto se enmarca en el área de Sistemas de Información e Ingeniería del Software, al abordar el diseño, desarrollo, despliegue y evaluación de una solución tecnológica orientada a resolver un problema real mediante una arquitectura escalable y modular.

1.1 MOTIVACIÓN

El sector de las gasolineras y los puntos de recarga eléctrica presentan una alta variabilidad en precios y disponibilidad, lo que genera incertidumbre en los usuarios y dificulta la toma de decisiones. Aunque existen herramientas que permiten consultar precios actuales, pocas soluciones integran funcionalidades avanzadas como predicción a corto plazo, recomendación basada en rutas o análisis geoespacial optimizado dentro de una misma plataforma.

La motivación principal de este proyecto es diseñar y desarrollar un sistema inteligente con interfaz web y despliegue en la nube que sea útil en la práctica y reúna los conocimientos más significativos adquiridos en el Grado en Ingeniería Informática. El sistema integra enfoques tecnológicos como analítica de datos, predicción, recomendación y asistencia inteligente con el fin de aportar un valor diferencial frente a las soluciones convencionales. El proyecto parte de un desarrollo académico previo, ampliando su alcance, mejorando la arquitectura y profundizando en la fundamentación técnica y metodológica exigida en un Proyecto Fin de Grado.

1.2 ESTRUCTURA DE LA MEMORIA

La memoria se organiza en los siguientes capítulos, siguiendo la estructura establecida por la normativa de Proyectos Fin de Grado del Grado en Ingeniería Informática de la Universidad de Deusto.

El **Capítulo 2, Antecedentes y Justificación**, analiza el estado del arte en plataformas de consulta de precios de combustibles y recarga eléctrica, identificando las carencias de las soluciones existentes y justificando la oportunidad del proyecto.

El **Capítulo 3, Marco Tecnológico**, describe las tecnologías seleccionadas para el desarrollo de la solución, incluyendo bases de datos, frameworks de desarrollo, servicios cloud y herramientas de machine learning.

El **Capítulo 4, Objetivos y Alcance**, define los objetivos específicos y medibles del proyecto, así como los límites del sistema, las restricciones asumidas y los aspectos excluidos del ámbito de este trabajo.

El **Capítulo 5, Planificación del Proyecto**, recoge la organización temporal, la descomposición en tareas e hitos, el encadenamiento de actividades y los diagramas de Gantt y plan de recursos humanos.

El **Capítulo 6, Presupuesto**, presenta la estimación económica del proyecto, desglosando los costes laborales por rol, los costes de material y equipo, y los servicios cloud utilizados.

El **Capítulo 7, Metodología**, justifica el enfoque de desarrollo ágil e iterativo adoptado y describe cómo se ha aplicado a lo largo del proyecto.

El **Capítulo 8, Desarrollo, Prueba y Despliegue**, constituye el núcleo técnico de la memoria, recogiendo la especificación de requisitos, el diseño arquitectónico, las decisiones tecnológicas, la implementación de los componentes y el plan de pruebas.

El **Capítulo 9, Valoración Ética del Proyecto**, analiza las implicaciones éticas y sociales de la solución, con atención al tratamiento de datos, la movilidad sostenible y la alineación con los Objetivos de Desarrollo Sostenible.

Finalmente, el **Capítulo 10, Conclusiones**, evalúa el grado de cumplimiento de los objetivos definidos y propone líneas de trabajo futuro. El documento se cierra con la bibliografía y los apéndices correspondientes.

2. CAPÍTULO SEGUNDO - ANTECEDENTES Y JUSTIFICACIÓN

2.1 SITUACIÓN ACTUAL DEL SECTOR DE GASOLINERAS

En los últimos años el sector energético vinculado a la movilidad ha experimentado una transformación impulsada por factores económicos, regulatorios y medioambientales. La volatilidad del precio de los combustibles fósiles, el crecimiento progresivo del parque de vehículos eléctricos y las políticas de transición energética han incrementado la necesidad de información precisa para la toma de decisiones de repostaje y recarga [3].

La coexistencia de distintas fuentes energéticas (gasolina, diésel, GLP, electricidad) introduce mayor complejidad en el proceso de decisión del usuario. Los precios presentan variaciones entre estaciones cercanas geográficamente y fluctuaciones temporales a corto plazo que el usuario raramente puede anticipar. Desde las administraciones públicas se ha impulsado la publicación de datos abiertos sobre precios de carburantes y localización de puntos de recarga [4, 5], facilitando el desarrollo de herramientas digitales de consulta. Sin embargo, como se analiza en las secciones siguientes, la oferta actual presenta limitaciones estructurales que justifican la propuesta de este trabajo.

2.2 ESTADO DEL ARTE

El estudio del estado del arte abarca tres ámbitos tecnológicos que convergen en la solución propuesta: a) la predicción de precios energéticos mediante aprendizaje automático, b) los sistemas de recomendación geoespacial aplicados a la movilidad y c) las arquitecturas *cloud* orientadas al procesamiento de datos en tiempo real.

2.2.1 Predicción de precios de combustibles

La predicción de precios en mercados energéticos es un problema ampliamente estudiado. Los enfoques clásicos basados en modelos estadísticos como ARIMA o GARCH han sido progresivamente desplazados por técnicas de aprendizaje automático que capturan relaciones no lineales y dependencias temporales complejas [6].

En el contexto de mercados energéticos, estudios recientes demuestran que los modelos de *gradient boosting* (en particular LightGBM) ofrecen un rendimiento superior en tareas de regresión sobre series temporales de precios al incorporar variables exógenas como el precio del crudo Brent o indicadores macroeconómicos [7].

Paralelamente, la regresión cuantílica permite estimar no solo el valor esperado del precio futuro, sino intervalos de confianza que habilitan una toma de decisiones más informada [8], enfoque adoptado en la presente plataforma.

Los modelos de *Deep Learning* como las redes LSTM han mostrado capacidad para modelar dependencias temporales a largo plazo [6], pero su mayor coste computacional y menor interpretabilidad los posicionan como alternativa secundaria frente a los modelos de *gradient boosting* en series temporales con datos tabulares y variables exógenas [7].

2.2.2 Sistemas de recomendación en movilidad

Los sistemas de recomendación aplicados a la movilidad han evolucionado desde soluciones basadas en proximidad geográfica hacia enfoques que integran múltiples criterios: coste, disponibilidad, tiempo de desvío y preferencias del usuario [9].

En el contexto de gasolineras, la recomendación basada en rutas implica la definición de un corredor geométrico alrededor del trayecto planificado y la evaluación de estaciones dentro de dicho corredor mediante una función de puntuación ponderada. Este enfoque, adoptado en la presente plataforma, minimiza el desvío total mientras optimiza el coste económico del repostaje o recarga. La integración de servicios de enrutamiento, como OpenRouteService (ORS) [10] a través de su API, es una práctica consolidada en aplicaciones de movilidad de código abierto, ya que permite ajustar la relación entre precisión y coste operativo.

2.2.3 Arquitecturas *cloud* para datos en tiempo real

Las arquitecturas de microservicios desplegadas en entornos *cloud-native* se han consolidado como el paradigma dominante para plataformas de datos a escala. Frente a las arquitecturas monolíticas, el modelo de microservicios ofrece desacoplamiento funcional, escalabilidad independiente por componente y resiliencia ante fallos parciales [11].

La adopción de formatos columnares como Apache Parquet [12] para la exportación de datos históricos y su almacenamiento en servicios de objeto como Google Cloud Storage es práctica habitual en *pipelines* de *machine learning* que requieren reproducibilidad y versionado del dato [8]. El despliegue *serverless* mediante contenedores en plataformas como Google Cloud Run [13] reduce la carga operativa de gestión de infraestructura y se alinea con los patrones de arquitectura adaptativa identificados por Gartner como prioritarios para el ciclo 2025–2026 [11].

2.2.4 Asistentes conversacionales en aplicaciones de dominio específico

El uso de modelos de lenguaje de gran escala (LLM) como motores de interfaces conversacionales en aplicaciones de dominio específico ha experimentado una adopción acelerada. En este proyecto no se emplea el *Model Context Protocol* (MCP), el sistema realiza peticiones HTTPS a la API de Gemini. Además, dispone de un conjunto de herramientas internas y flujos de orquestación que automatizan la toma de decisiones y la composición de acciones en respuesta a consultas abiertas. Estas herramientas coordinan llamadas a las APIs del sistema, consultas geoespaciales a la base de datos y la invocación de modelos de predicción cuando procede; por ejemplo, para responder a consultas como “¿cuál es la gasolinera más barata?” el sistema puede combinar: a) una búsqueda por proximidad mediante PostGIS sobre la tabla `gasolineras`, b) la recuperación de precios actuales y su histórico en `precios_historicos`, y c) la consulta al servicio de predicción (modelo LightGBM con regresión cuantílica) para estimar tendencias a corto plazo.

Estas capacidades se apoyan en la arquitectura descrita en el Capítulo 8: un API Gateway (Hono) centraliza las peticiones, varios microservicios gestionan dominios concretos (gasolineras, electrolineras, usuarios, recomendación y predicción) y la capa de persistencia (PostgreSQL/Neon con PostGIS) permite consultas eficientes y reproducibles. El servicio de sincronización de datos mantiene la frescura mediante trabajos programados (Cloud Scheduler / Cloud Run Jobs) que exportan snapshots en formato Parquet para el pipeline de ML, mientras que en entornos locales la orquestación se realiza con Docker Compose.

Adicionalmente, el sistema integra un motor de recomendación por ruta que combina trazados de OpenRouteService con una función de puntuación ponderada (coste, desvío y preferencias del usuario), y un asistente conversacional apoyado por un LLM (consultas HTTPS a la API externa) que orquesta llamadas internas para componer respuestas enriquecidas. Para mejorar la disponibilidad, el *gasolineras-service* implementa mecanismos de fallback en memoria y políticas de refresco controladas por configuración, evitando latencias impredecibles en peticiones de lectura.

En conjunto, estos elementos convierten consultas simples en flujos compuestos y reproducibles, integrando datos abiertos, capacidad predictiva y servicios en la nube para ofrecer respuestas contextualizadas y accionables al usuario.

2.3 SOLUCIONES EXISTENTES Y LIMITACIONES

2.3.1 Plataformas gubernamentales

El ecosistema de herramientas oficiales en España ha evolucionado significativamente en los últimos meses. El *Geoportal de Hidrocarburos* del MITECO [5] constituye la fuente de referencia para precios de carburantes, con API REST de acceso público [4]. Su interfaz web permite la consulta de precios actuales pero carece de análisis histórico, capacidad predictiva y recomendación basada en ruta.

La aplicación móvil **Ruta-e**, evolución de la anterior GeoGasolineras, integra en una sola plataforma datos de estaciones de servicio y puntos de recarga eléctrica, con filtros por tipo de combustible, precio y horario. Supone un avance respecto a la fragmentación previa, aunque sigue siendo una herramienta de consulta estática sin capacidad analítica ni predictiva.

En el ámbito de la recarga eléctrica, el **mapa REVE** (Red de Puntos de Recarga para Vehículos Eléctricos), desarrollado por Red Eléctrica por mandato del MITECO e impulsado bajo el Real Decreto-ley 4/2024, ofrece desde abril de 2025 información dinámica en tiempo real sobre disponibilidad y tarifas de más de 40.000 puntos de recarga. En diciembre de 2025 lanzó su aplicación móvil con planificador de rutas básico para vehículos eléctricos. Se trata de la iniciativa oficial más avanzada en este ámbito, aunque centrada exclusivamente en recarga eléctrica y sin integración con combustibles convencionales ni capacidad predictiva.

2.3.2 Aplicaciones de terceros

Existen aplicaciones ampliamente utilizadas como *Gasolineras España*, *Waze* (con integración parcial de precios) o *GasBuddy* en otros mercados anglosajones, que abordan parcialmente la búsqueda por proximidad. Estas soluciones presentan limitaciones estructurales comunes: dependencia de datos *crowdsourced* con la consiguiente latencia y sesgo potencial, ausencia de modelos predictivos y recomendación por ruta limitada a criterios de proximidad simple.

2.3.3 Síntesis comparativa

La Tabla 2.1 resume las capacidades de las principales soluciones existentes frente al sistema propuesto.

La síntesis pone de manifiesto que, si bien las iniciativas oficiales recientes (Ruta-e, REVE) han reducido la fragmentación entre energías, ninguna plataforma integra análisis histórico, predicción de precios y recomenda-

Cuadro 2.1: Comparativa de funcionalidades entre soluciones existentes y TankGo

Funcionalidad	Geoportal	Ruta-e	REVE	Apps terceros	TankGo
Precios combustible en tiempo real	✓	✓	-	✓	✓
Historial de precios	-	-	-	Parcial	✓
Predicción de precios	-	-	-	-	✓
Recomendación por ruta	-	✓	Parcial EV	Parcial	✓
Integración combustible + EV	-	✓	-	-	✓
Asistente conversacional	-	-	-	-	✓
API pública documentada	✓	-	-	-	✓
Arquitectura <i>cloud-native</i>	-	-	-	-	✓

ción de ruta con soporte conversacional en un único sistema. La propuesta de valor de TankGo reside precisamente en esa integración, que transforma datos de consulta en un sistema de apoyo a la decisión.

2.4 ORIGEN DEL PROYECTO Y EVOLUCIÓN

El presente proyecto parte de un prototipo académico desarrollado por el autor en la asignatura *Desarrollo Avanzado de Aplicaciones para la Web de Datos* [1]. Dicho sistema permitía la consulta de precios mediante la federación de fuentes de datos abiertos y una visualización geoespacial básica, con una arquitectura acotada a objetivos formativos.

El actual Trabajo de Fin de Grado transforma esa base en una plataforma profesional bajo tres ejes estratégicos alineados con tendencias recientes de la industria [11]:

1. **Infraestructura escalable:** migración hacia arquitectura *cloud-native* con capacidad de procesamiento de datos masivos e ingesta en tiempo real, capacidad crítica para mitigar riesgos en el sector energético [8].
2. **Inteligencia de datos:** incorporación de un motor de predicción de precios basado en LightGBM con regresión cuantílica, seleccionado por su equilibrio entre precisión, coste computacional e interpretabilidad [7, 8].
3. **Interacción conversacional:** integración de un asistente de voz, que habilita consultas en lenguaje natural sobre datos en tiempo real sin modificar la arquitectura subyacente.

Con este enfoque, el sistema deja de ser una herramienta meramente informativa y se convierte en una plataforma web de apoyo a la decisión, capaz de gestionar, analizar y visualizar datos.

2.5 JUSTIFICACIÓN TÉCNICA Y ACADÉMICA

Desde el punto de vista **técnico**, el proyecto integra múltiples disciplinas de la Ingeniería Informática: arquitectura de sistemas *cloud-native*, gestión de datos y series temporales, inteligencia computacional aplicada [7] e ingeniería de interfaces con visualización geoespacial y asistencia conversacional.

Desde la **perspectiva académica**, este trabajo se justifica como síntesis integral de las competencias del Grado en Ingeniería Informática. El proyecto demuestra madurez metodológica al fundamentar las decisiones de diseño en el análisis del estado del arte [11], aplicar planificación formal con estimación de costes y evaluar de forma crítica las limitaciones del sistema y sus líneas de evolución futura.

2.5.1 Alineación con los Objetivos de Desarrollo Sostenible

La Agenda 2030 [14] establece el marco global de referencia en sostenibilidad. La plataforma se alinea con dos objetivos directamente relevantes para el ámbito de la movilidad inteligente:

- **ODS 7 (Energía asequible y no contaminante):** al integrar en una única interfaz precios de combustibles y puntos de recarga eléctrica, la plataforma facilita la transición hacia modelos de movilidad más sostenibles y promueve la eficiencia energética en el transporte por carretera.
- **ODS 9 (Industria, innovación e infraestructura):** la transformación de fuentes de datos heterogéneas en un sistema de soporte a la decisión mediante *cloud computing* e inteligencia artificial promueve infraestructuras digitales resilientes e innovadoras [9], en línea con los principios de la Agenda Digital española [15].

3. CAPÍTULO TERCERO - MARCO TECNOLÓGICO

La elección de las tecnologías que dan soporte a TankGo responde a un análisis previo de alternativas en el que se han ponderado criterios de rendimiento, madurez del ecosistema, compatibilidad con el modelo de despliegue *cloud-native*, coste operativo y alineación con las competencias adquiridas durante el grado. Este capítulo presenta las tecnologías seleccionadas organizadas por capa arquitectónica; la justificación técnica detallada de cada decisión se recoge en el Capítulo 8.

3.1 CAPA DE PRESENTACIÓN: REACT Y LEAFLET.JS

El cliente web está desarrollado con **React 19** [16], biblioteca de interfaz de usuario de Meta basada en componentes reutilizables y DOM virtual. Frente a Vue.js y Angular [17, 18], React ofrece el ecosistema cartográfico más maduro a través de `react-leaflet` [19] y cuenta con la mayor adopción en el mercado laboral, alcanzando el 44,7 % según la encuesta de Stack Overflow de 2025 [20].

Para la visualización geoespacial se emplea **Leaflet.js**, biblioteca de código abierto bajo licencia BSD-2 con un ecosistema de *plugins* extenso [21]. La API de Google Maps queda descartada por su modelo de precios, y Mapbox GL JS por el cambio a licencia propietaria en 2020 [22]. El soporte de agrupación de marcadores mediante `leaflet.markercluster` [23] resulta esencial para gestionar las más de 11.000 estaciones del catálogo.

El frontend está concebido como **Progressive Web App (PWA)**, con *service worker* gestionado por Workbox [24, 25, 26] para soporte offline limitado e instalación en dispositivos móviles. La internacionalización en castellano, euskera e inglés se implementa mediante `i18next` [27, 28].

3.2 CAPA DE GATEWAY: HONO SOBRE NODE.JS

El API Gateway utiliza **Hono** [29], framework web ultraligero con compatibilidad nativa para múltiples runtimes y un rendimiento cercano a 350.000 req/s en benchmarks independientes [30]. Frente a Express.js [31], Hono adopta la especificación Fetch API del estándar web, lo que le confiere un modelo de concurrencia asíncrono más adecuado para un gateway cuya carga es mayoritariamente I/O-bound.

3.3 CAPA DE SERVICIOS DE NEGOCIO: FASTAPI Y FASTIFY

Los servicios Python (`gasolineras-service`, `recomendacion-service` y `prediction-service`) utilizan **FastAPI** [32, 33], construido sobre Starlette [34] y Pydantic v2 [35]. Su arquitectura ASGI ofrece un rendimiento superior a Flask [36] y Django [37] en escenarios de API REST, y genera documentación OpenAPI de forma automática [38], lo que facilita la agregación de especificaciones en el gateway.

El servicio de usuarios está implementado con **Fastify**, cuyo ecosistema de plugins para JWT y cookies `httpOnly` (`@fastify/jwt`, `@fastify/cookie`) y su validación basada en JSON Schema lo hacen especialmente adecuado para la gestión de identidad y autenticación.

3.4 CAPA DE DATOS: POSTGRESQL Y APACHE PARQUET

Todos los servicios con necesidades de persistencia utilizan **PostgreSQL**, alojado en **Neon** [39], plataforma de PostgreSQL serverless que escala a cero en inactividad. La extensión **PostGIS** habilita las consultas geoespaciales (`ST_DWithin`, `ST_LineLocatePoint`, `ST_Distance`) requeridas por el servicio de recomendación.

Los datos históricos exportados para el entrenamiento de modelos se almacenan en **Apache Parquet** [12], formato columnar que reduce el volumen de I/O al leer únicamente las columnas necesarias y produce ficheros entre un 80 y un 90 % más pequeños que sus equivalentes CSV [40, 41]. Los ficheros se almacenan en Google Cloud Storage y se leen desde `prediction-service` mediante PyArrow.

3.5 CAPA DE INTELIGENCIA: LIGHTGBM

El módulo de predicción utiliza **LightGBM** en modo de regresión cuantílica (percentiles 0,05, 0,50 y 0,95). Los modelos de *gradient boosting* ofrecen un rendimiento superior a ARIMA y LSTM en series temporales con datos tabulares y variables exógenas [7], y producen directamente intervalos de confianza sin asumir distribuciones sobre los errores [8]. Su bajo coste de entrenamiento es compatible con la ejecución semanal del *job* en un entorno serverless con límites de memoria y CPU.

3.6 CAPA DE ENRUTAMIENTO GEOESPACIAL: OPENROUTESERVICE

Para el cálculo de trayectos y matrices de distancias se integra **OpenRouteService** (ORS) [42], plataforma de enrutamiento de código abierto con API REST gestionada. OSRM [10], aunque más rápido en tiempo de respuesta, requiere un servidor propio con requisitos de RAM elevados para el grafo viario español, lo que resulta inviable en Cloud Run. ORS expone las funcionalidades de *directions* y *matrix* necesarias para el algoritmo de recomendación sin infraestructura adicional, e incorpora la evitación de peajes como parámetro nativo.

3.7 CAPA DE CARTOGRAFÍA: OPENSTREETMAP Y NOMINATIM

Las teselas del mapa y la geocodificación se obtienen de **OpenStreetMap** y su motor de búsqueda **Nominatim**. El API Gateway actúa como proxy hacia Nominatim, centralizando la caché y evitando bloqueos CORS. La API de Geocoding de Google y la de Mapbox quedan descartadas por introducir dependencia de servicios propietarios con modelos de precios variables.

3.8 CAPA DE INTELIGENCIA CONVERSACIONAL: GEMINI Y GOOGLE GENAI

El asistente de voz implementa un pipeline **STT** → **LLM con tool calling** → **TTS** sobre **Gemini 2.5 Flash** [43], accedido mediante llamadas HTTP REST con la biblioteca `@google/genai`. El uso de un único modelo multimodal simplifica la arquitectura, reduce la latencia acumulada entre etapas y elimina la gestión de múltiples claves de API [11]. El *tool calling* [44, 45] expone dos herramientas al modelo: `get_prices` y `get_nearest_stations`, que delegan en los endpoints HTTP del gateway.

3.9 PLATAFORMA DE DESPLIEGUE: GOOGLE CLOUD

El despliegue en producción utiliza **Cloud Run** [13] como plataforma serverless de ejecución de contenedores, **Cloud Build** [46] para los pipelines CI/CD (siete ficheros `cloudbuild-*.yaml`, uno por servicio), **Cloud Storage** [47] para los artefactos del pipeline de ML y **Secret Manager** para la inyección de credenciales en tiempo de despliegue. Frente a AWS, GCP ofrece integración nativa con Gemini y las APIs de Google, simplificando la gestión de identidades mediante IAM [11].

3.10 SÍNTESIS: JUSTIFICACIÓN DEL STACK TECNOLÓGICO

La Tabla 3.1 resume las tecnologías empleadas, su capa de aplicación y el criterio principal que motivó su elección frente a las alternativas evaluadas.

Cuadro 3.1: Resumen del stack tecnológico de TankGo

Tecnología	Capa	Alternativas evaluadas	Criterio de elección
React 19	Frontend	Vue.js, Angular	Ecosistema Leaflet, adopción
Leaflet.js	Frontend	Google Maps, Mapbox	Licencia abierta, sin coste
Hono 4.10	Gateway	Express.js, Fastify	Rendimiento async, WS nativo
FastAPI 0.115	Backend Python	Flask, Django	ASGI, OpenAPI automático
Fastify 5.7	Backend Node.js	Express.js, NestJS	JWT/cookie ecosystem
PostgreSQL / Neon	Datos	MySQL, MongoDB	PostGIS, serverless, ACID
Apache Parquet	ML pipeline	CSV, JSON, Avro	Columnar, compresión, Python
LightGBM 4.3	ML modelo	XGBoost, ARIMA, LSTM	Velocidad, quantile regression
ORS (OpenRouteService)	Ruteo	OSRM, Google Directions	API REST gestionada, Matrix
OpenStreetMap	Cartografía	Google Maps, Mapbox	Datos libres, sin coste API
Nominatim	Geocodificación	Google Geocoding	Gratuito, integrado con OSM
Gemini 2.5 Flash	IA / Voz	GPT-4o + Whisper, Gemini Pro	Multimodal, latencia baja
Google Cloud Run	Despliegue	AWS Lambda, Azure Container	Escala a cero, integración GCP

El conjunto de tecnologías elegidas comparte un denominador común: todas son de uso libre o tienen niveles gratuitos generosos, cuentan con documentación oficial exhaustiva y comunidades activas. Este criterio de *sostenibilidad tecnológica* ha primado sobre la mera novedad, garantizando que la solución pueda ser mantenida y extendida con independencia de los ciclos comerciales de proveedores específicos.

4. CAPÍTULO CUARTO - OBJETIVOS Y ALCANCE

Este capítulo concreta los objetivos que guían el desarrollo del proyecto y delimita el alcance del sistema resultante. Se definen tanto el objetivo general como los objetivos específicos y medibles, siguiendo criterios de claridad, relevancia y viabilidad. Además, se establecen las limitaciones conocidas y las exclusiones explícitas para evitar ambigüedades en la evaluación del trabajo realizado.

4.1 OBJETIVO GENERAL

Diseñar, desarrollar y desplegar una plataforma software *cloud-native* que integre, en un único sistema, la consulta y comparación de precios de combustibles y puntos de recarga eléctrica en España, incorporando capacidades de análisis histórico, predicción de precios a corto plazo, recomendación de repostaje basada en rutas y asistencia conversacional, con el fin de transformar datos energéticos dispersos en conocimiento accionable y accesible para el usuario final.

4.2 OBJETIVOS ESPECÍFICOS Y MEDIBLES

Los objetivos específicos desglosan el objetivo general en metas concretas, verificables y acotadas en el tiempo, siguiendo el criterio SMART (*Specific, Measurable, Achievable, Relevant, Time-bound*) [48]. Se organizan en torno a los cinco ejes funcionales del sistema:

4.2.1 Ingesta y gestión de datos

- **OE-01.** Implementar un pipeline de sincronización automática con la API REST del Ministerio de Industria (MinITUR) que garantice la disponibilidad de datos de precios actualizados con una frecuencia mínima diaria, con persistencia histórica en base de datos relacional.
- **OE-02.** Integrar una fuente de datos de puntos de recarga eléctrica en el mismo modelo de datos que los combustibles convencionales, permitiendo consultas unificadas sobre ambas fuentes energéticas.
- **OE-03.** Diseñar e implementar un pipeline de exportación de datos históricos en formato Apache Parquet hacia Google Cloud Storage, como base de datos de entrenamiento reproducible para los modelos de aprendizaje automático.

4.2.2 Consulta y visualización geoespacial

- **OE-04.** Desarrollar una interfaz web progresiva (PWA) con mapa interactivo que permita la búsqueda y filtrado de estaciones por provincia, municipio, marca y tipo de combustible e internacionalización en tres idiomas (español, inglés y euskera).
- **OE-05.** Implementar la visualización del historial de precios por estación mediante gráficos temporales interactivos.

4.2.3 Predicción de precios

- **OE-06.** Desarrollar e integrar un servicio de predicción de precios basado en modelos de *gradient boosting* (LightGBM) con regresión cuantílica, incorporando el precio del crudo Brent como variable exógena [7], con capacidad de generar predicciones individualizadas por estación de servicio en un periodo de 7 días.
- **OE-07.** Evaluar comparativamente el rendimiento del modelo seleccionado frente a alternativas (XGBoost, regresión lineal como *baseline*) mediante métricas estándar de regresión (MAE, RMSE, R^2) [8].

4.2.4 Recomendación basada en rutas

- **OE-08.** Implementar un servicio de recomendación de gasolineras para rutas A→B que identifique las estaciones óptimas dentro del corredor geográfico de la ruta, evalúe las candidatas según una función de puntuación ponderada que combine precio del combustible y desvío respecto al trayecto original, y devuelva al usuario un ranking de recomendaciones con estimación del ahorro económico frente a la opción menos favorable.
- **OE-09.** Integrar un servicio de enrutamiento externo como backend único de cálculo de rutas en el entorno de producción cloud, garantizando la resiliencia del sistema ante la indisponibilidad temporal de la API externa mediante mecanismos de control de concurrencia y fallback configurable.

4.2.5 Arquitectura, despliegue y calidad

- **OE-10.** Diseñar e implementar una arquitectura de microservicios con API Gateway centralizado, donde cada servicio sea independientemente desplegable y escalable, siguiendo los principios de las arquitec-

turas cloud-native [11].

- **OE-11.** Desplegar la totalidad de la plataforma en Google Cloud Run mediante pipelines de integración y entrega continua (CI/CD) con Google Cloud Build, con un mínimo de siete pipelines independientes (uno por servicio).
- **OE-12.** Implementar un sistema de autenticación seguro basado en JWT almacenado en *httpOnly cookies*, con soporte para autenticación OAuth 2.0 mediante Google y gestión de perfiles de usuario persistentes.
- **OE-13.** Desarrollar e integrar un asistente de voz basado en un pipeline STT→LLM→TTS implementado sobre la API REST de Google GenAI (Gemini 2.5 Flash) [43], con capacidad de *tool calling* [44] para consultar precios y estaciones cercanas en tiempo real a través del API Gateway del sistema.

4.3 ALCANCE DEL SISTEMA

El alcance define los límites funcionales y técnicos del sistema desarrollado, estableciendo qué está incluido en la solución entregada.

4.3.1 Ámbito geográfico y de datos

El sistema cubre la totalidad del territorio español continental e insular, utilizando como fuente primaria de datos de carburantes la API pública del Ministerio de Industria, Turismo y Comercio (MinITUR) [4], que publica precios de aproximadamente 11.000 estaciones de servicio activas en España. Los puntos de recarga eléctrica se obtienen del portal mapareve.es, complementando la oferta de datos de movilidad eléctrica disponible institucionalmente [9].

4.3.2 Componentes funcionales incluidos

La plataforma entregada comprende los siguientes componentes funcionales:

1. **Servicio de gasolineras:** ingesta, normalización, persistencia y consulta de precios de carburantes con historial temporal. Ofrece búsqueda geoespacial por radio, filtrado por provincia, municipio, marca y tipo de combustible, e integra los puntos de recarga eléctrica en el mismo modelo de datos.
2. **Servicio de usuarios:** registro, autenticación (tradicional y OAuth Google), gestión de perfiles con prefe-

rencias de combustible y vehículo, y sistema de favoritos sincronizado. Emite credenciales mediante JWT almacenado en *httpOnly cookie* y expone endpoints internos para la comunicación entre servicios.

3. **Servicio de recomendación:** cálculo de las gasolineras óptimas para un trayecto A→B definido por el usuario, mediante identificación de candidatas en el corredor geográfico de la ruta, cálculo de desvíos reales respecto al trayecto original, evaluación mediante función de puntuación ponderada por precio y desvío, y presentación de un ranking con estimación del ahorro económico. Soporta configuración del tipo de combustible, desvío máximo aceptable y capacidad del depósito, e integra un backend de enrutamiento externo para el cálculo preciso de distancias y tiempos de desvío.
4. **Servicio de predicción:** generación de predicciones de precio a corto plazo por estación, basadas en modelos de *gradient boosting* entrenados sobre series históricas de precios. Incorpora el precio del crudo Brent como variable exógena y cubre las estaciones con historial suficiente y mayor demanda de uso.
5. **Asistente de voz:** interfaz conversacional en lenguaje natural que permite al usuario consultar precios y estaciones cercanas mediante voz. Implementa un pipeline STT→LLM→TTS con capacidad de *tool calling* para acceder en tiempo real a los datos del sistema a través del API Gateway.
6. **API Gateway:** punto de entrada único para el frontend, que actúa como proxy reverso hacia los servicios internos, agrega la documentación OpenAPI de todos ellos, gestiona la autenticación mediante cookies y sirve de bridge WebSocket para el asistente de voz.
7. **Frontend PWA:** aplicación web progresiva instalable con mapa interactivo, historial de precios, gestión de favoritos, recomendación de ruta y asistente de voz integrado. Ofrece soporte *offline*, modo oscuro e internacionalización en tres idiomas [24, 27].

4.3.3 Entorno de despliegue

La plataforma se despliega íntegramente en Google Cloud Platform (GCP) [13], utilizando Cloud Run como servicio de ejecución de contenedores serverless, Artifact Registry [49] para el almacenamiento de imágenes Docker [50], Cloud Build [46] para la automatización del pipeline CI/CD y Secret Manager [51] para la gestión segura de credenciales en producción. La base de datos relacional se externaliza en Neon (PostgreSQL gestionado) y el almacenamiento de objetos para el pipeline de ML se realiza en Google Cloud Storage [47].

4.4 LIMITACIONES

Las limitaciones recogen aquellos aspectos que, estando dentro del alcance conceptual del proyecto, presentan restricciones técnicas o de recursos conocidas en la versión entregada.

- **Frecuencia de actualización de precios:** La API del Ministerio de Industria publica datos con una frecuencia que puede variar entre actualizaciones horarias y diarias según la estación, por lo que el sistema no puede garantizar precios en tiempo real estricto, sino *near real-time* con un desfase máximo determinado por la fuente oficial.
- **Cobertura del modelo predictivo:** La generación y mantenimiento de modelos predictivos por estación es intensiva en recursos computacionales (entrenamiento, almacenamiento de artefactos y ejecución de inferencia periódica) y depende de la disponibilidad de series temporales suficientemente largas y de calidad. Por ello, el servicio de predicción limita su cobertura a las estaciones que disponen de historial suficiente y a las marcadas como favoritas por los usuarios, priorizando la relación coste/beneficio y la utilidad práctica frente a una cobertura completa que sería costosa e innecesaria para estaciones con datos escasos. El diseño detallado del pipeline de entrenamiento, la política de refresco y las estimaciones de coste computacional se documentan en el Capítulo 8 (Sección 8.3).
- **Precisión de la recomendación por ruta:** La calidad de la recomendación depende de la disponibilidad y precisión del backend de enrutamiento seleccionado. El fallback a distancia en línea recta, activable mediante configuración, reduce la precisión del corredor geométrico calculado.
- **Escalabilidad del asistente de voz:** El servicio de voz está sujeto a los límites de cuota de la API de Gemini y al rate limiting configurado (40 peticiones por minuto por IP), lo que limita su uso concurrente intensivo.
- **Disponibilidad de datos de recarga eléctrica:** La información de puntos de recarga eléctrica procede de una fuente de terceros (mapareve.es) cuya disponibilidad y completitud no están garantizadas de forma institucional, a diferencia de los datos de carburantes del Ministerio.

4.5 EXCLUSIONES

Las exclusiones delimitan explícitamente lo que queda fuera del alcance del presente proyecto, evitando ambigüedades en la evaluación del trabajo realizado.

- **Aplicación móvil nativa:** El sistema se entrega como PWA instalable, pero no se desarrolla una aplicación nativa para iOS o Android. La PWA cubre los casos de uso de movilidad mediante el navegador del dispositivo.
- **Integración con navegadores GPS:** No se integra con sistemas de navegación en tiempo real tipo Google Maps o Waze para guía turn-by-turn. La recomendación de ruta se limita al cálculo de la estación óptima, no a la navegación posterior. Sin embargo, la aplicación proporciona enlaces directos a Google Maps para la navegación hacia la estación recomendada.
- **Pagos y reservas:** El sistema no contempla funcionalidades de pago, reserva de surtidores ni integración con sistemas de fidelización de las cadenas de gasolineras.
- **Predicción de precios de electricidad:** El módulo predictivo se centra exclusivamente en combustibles líquidos (gasolina y diésel). La predicción del coste de recarga eléctrica, que depende de tarifas horarias (PVPC) y contratos de operador, queda fuera del alcance de esta versión.
- **Soporte multipaís:** La plataforma está diseñada y configurada para el mercado español. La extensión a otros mercados europeos requeriría adaptaciones en las fuentes de datos, la normalización de formatos y la cobertura geográfica de los servicios de enrutamiento.
- **Panel de administración:** No se desarrolla una interfaz de administración gráfica para la gestión de usuarios, sincronizaciones o modelos ML. La administración se realiza mediante endpoints protegidos por *internal secret* y acceso directo a la base de datos.

5. CAPÍTULO QUINTO - PLANIFICACIÓN DEL PROYECTO

Este capítulo describe cómo se organizó y gestionó el desarrollo de TankGo a lo largo de sus dos fases. Se detalla la descomposición del trabajo en tareas, la estimación de tiempos, el cronograma seguido, los hitos alcanzados y las replanificaciones que surgieron durante el proceso.

5.1 DESCOMPOSICIÓN EN TAREAS (EDT)

La Estructura de Descomposición del Trabajo (*Work Breakdown Structure*, WBS) organiza el proyecto en bloques funcionales que reflejan la evolución real del desarrollo. Se distinguen dos grandes fases, cada una subdividida en paquetes de trabajo.

Fase 1 - Prototipo inicial (octubre-diciembre 2025)

1. **Definición del alcance y arquitectura inicial.** Estudio de fuentes de datos disponibles, definición de objetivos del sistema y diseño de la arquitectura de microservicios.
2. **Módulo de gasolineras.** Integración con la API del Ministerio para obtener datos de precios en tiempo real, diseño del modelo de datos y desarrollo del microservicio correspondiente.
3. **Módulo de usuarios.** Implementación del sistema de autenticación, gestión de perfiles y preferencias.
4. **API Gateway.** Configuración del gateway como punto de entrada unificado para los distintos microservicios.
5. **Interfaz de usuario (frontend).** Diseño UX/UI y desarrollo de la aplicación web para consulta y comparación de precios.
6. **Primer despliegue (Render PaaS).** Puesta en producción del sistema en la plataforma Render como validación del funcionamiento integral.

Fase 2 - Ampliación como PFG (enero-mayo 2026)

7. **Migración y mejora del despliegue a GCP.** Reconfiguración de la infraestructura en Google Cloud Platform, incluyendo Cloud Run, Cloud Storage, así como mantener la base de datos de Neon Tech.

8. **Módulo de electromovilidad.** Integración de puntos de recarga de vehículos eléctricos como extensión del dominio de datos del sistema.
9. **Sistema de recomendación de rutas.** Desarrollo del módulo de rutado que sugiere paradas de repostaje óptimas en función de la ruta del usuario.
10. **Módulo de predicción de precios (ML).** Entrenamiento y despliegue del modelo LightGBM para la estimación de precios futuros de carburante.
11. **Asistente conversacional.** Implementación del agente de voz para la interacción en lenguaje natural con el sistema.
12. **Mejora y refinamiento del sistema de rutado.** Optimización de los algoritmos de recomendación y validación con casos de uso reales.
13. **Documentación y memoria del PFG.** Redacción de la memoria técnica, preparación de la presentación y revisión final.

5.2 ESTIMACIÓN DE TIEMPOS

La estimación de tiempos se realizó de forma incremental: la Fase 1 contó con una dedicación intensiva propia de una asignatura de proyecto, mientras que la Fase 2 se desarrolló a un ritmo de entre 10 y 15 horas semanales, compatibilizando el trabajo con otras actividades académicas y la colaboración con el grupo de investigación DEUSTEK/MORElab [52] de la Facultad de Ingeniería de la Universidad de Deusto.

La tabla siguiente recoge la estimación de horas por paquete de trabajo:

Cuadro 5.1: Estimación de horas por paquete de trabajo

Fase	Paquete de trabajo	Horas estimadas
Fase 1	Definición del alcance y arquitectura	15
	Módulo de gasolineras	30
	Módulo de usuarios	20
	API Gateway	15
	Frontend	45
	Despliegue en Render	10
Fase 2	Migración a GCP	25
	Módulo de electromovilidad	10
	Sistema de recomendación de rutas	50
	Módulo de predicción (ML)	40
	Asistente conversacional	35
	Refinamiento del rutado	25
	Refinamiento de la interfaz	15
	Correcciones y mejoras	25
	Documentación y memoria	40
Total estimado		400

Las estimaciones recogidas en la tabla constituyen una aproximación retrospectiva al esfuerzo real invertido, elaborada a partir del registro de tareas y del seguimiento del tablero Kanban. La naturaleza incremental del proyecto hace que la estimación prospectiva de horas presente mayor incertidumbre que en proyectos con alcance cerrado desde el inicio; por ello, las cifras reflejan el tiempo efectivamente dedicado con mayor fidelidad que una planificación previa al desarrollo.

5.3 CRONOGRAMA (DIAGRAMA DE GANTT)

El cronograma del proyecto abarca el período comprendido entre octubre de 2025 y mayo de 2026. La figura 5.1 representa el diagrama de Gantt con las principales tareas del proyecto distribuidas en el tiempo.

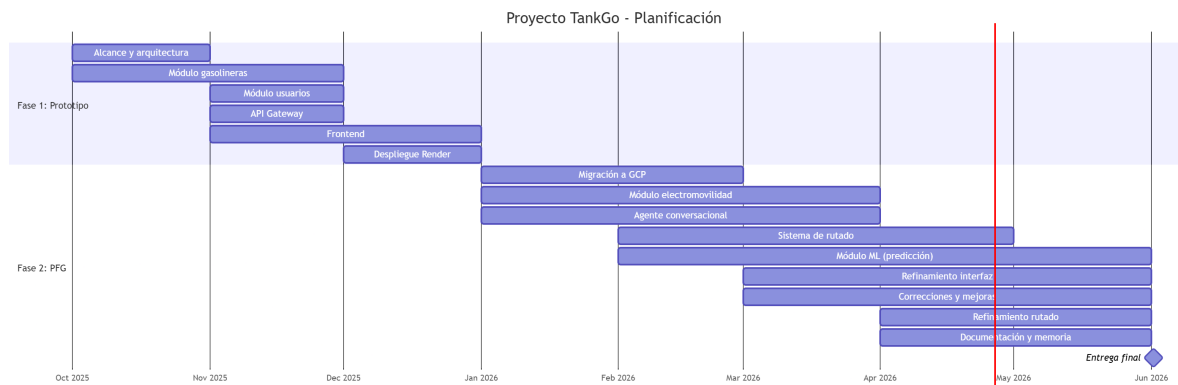


Figura 5.1: Diagrama de Gantt del proyecto TankGo

5.4 HITOS DEL PROYECTO

A lo largo del proyecto se definieron tres hitos principales que marcaron el cierre de etapas significativas y sirvieron como puntos de validación con el director:

M1 - Prototipo funcional (diciembre 2025). Finalización de la primera versión operativa del sistema, con los módulos de gasolineras, usuarios, gateway y frontend integrados y desplegados en Render. Este hito cerró la fase correspondiente a la asignatura de proyecto y sentó las bases del PFG.

M2 - Sistema completo en GCP (mayo 2026). Finalización de todos los módulos avanzados: migración a Google Cloud Platform, sistema de recomendación de rutas, módulo de predicción de precios con LightGBM y asistente conversacional. Este hito supuso la validación funcional del sistema en su versión completa.

M3 - Entrega del PFG (junio 2026). Cierre del proyecto con la entrega de la memoria técnica y la presentación ante el tribunal. Este hito incluye la revisión final de la documentación y la preparación de la defensa.

5.5 ENCADENAMIENTO DE ACTIVIDADES

El encadenamiento de actividades refleja las dependencias lógicas entre los distintos paquetes de trabajo, es decir, qué tareas deben completarse antes de que otras puedan comenzar. Dado que el proyecto siguió un enfoque incremental, muchas dependencias son secuenciales dentro de cada módulo, aunque algunas tareas de

la segunda fase podían abordarse en paralelo una vez estabilizado el núcleo del sistema.

La **definición del alcance y la arquitectura** constituye la actividad de partida del proyecto: ningún módulo puede iniciarse sin tener clara la estructura general del sistema y las fuentes de datos a integrar. A partir de ella, el **módulo de gasolineras** y el **módulo de usuarios** pueden desarrollarse en paralelo, ya que ambos son funcionalmente independientes. El primero es el componente central de la plataforma y su finalización es prerequisite para el desarrollo del frontend, que consume los datos que expone. El **API Gateway**, por su parte, se desarrolló de forma incremental conforme se añadían nuevos microservicios, requiriendo que al menos uno de ellos estuviera operativo para poder configurar el enrutamiento. El **frontend** no puede estabilizarse hasta que tanto el gateway como los módulos principales expongan contratos de API consolidados. El **despliegue en Render** cierra la Fase 1 y requiere que todos los componentes anteriores estén integrados y probados.

La **migración a GCP** abre la Fase 2 y es prerequisite para cualquier componente avanzado, ya que la nueva infraestructura proporciona los servicios de cómputo, base de datos y orquestación sobre los que se apoya el resto del sistema. Una vez completada, el **módulo de electromovilidad** y el **agente conversacional** pueden desarrollarse en paralelo, al tratarse de extensiones independientes. El **sistema de recomendación de rutas** depende de que los datos de gasolineras y puntos de recarga estén disponibles y correctamente modelados, mientras que el **módulo de predicción** (LightGBM) requiere además disponer de series históricas de precios almacenadas en GCP. El **refinamiento del rutado** se inicia una vez que el sistema de recomendación está operativo y se han identificado sus limitaciones mediante pruebas. Finalmente, la **documentación y memoria** se elabora en paralelo con el desarrollo técnico de la Fase 2, aunque con una intensidad creciente hacia el cierre del proyecto.

5.6 PLAN DE RECURSOS HUMANOS

El proyecto TankGo ha sido desarrollado íntegramente por un único alumno, con la supervisión periódica del director del PFG. Los dos roles principales son el del alumno como desarrollador y el del director como tutor académico, a los que se suma una colaboración externa de carácter consultivo con el grupo de investigación DEUSTEK/MORElab [52] de la Universidad de Deusto.

El **alumno** ha asumido la responsabilidad íntegra del diseño, implementación, pruebas y documentación del sistema, así como las tareas propias de gestión del proyecto: planificación de tareas, seguimiento del tablero Kanban y coordinación con los agentes externos. La dedicación fue de aproximadamente 10–15 horas semanales durante la Fase 2.

El **director del PFG** ha ejercido la supervisión técnica y académica del proyecto mediante reuniones periódicas de seguimiento, orientación en las decisiones de diseño y revisión de los entregables, con una dedicación estimada de 1–2 horas semanales a lo largo de todo el proyecto.

Adicionalmente, el proyecto ha contado con la colaboración del grupo de investigación **DEUSTEK/MORElab**[52], a través del proyecto *NextEdge*, de la Universidad de Deusto, dirigido por el Dr. Diego López-de-Ipiña y la Dra. Oihane Gómez Carmona. El grupo mostró un interés activo en la solución y ha promovido el desarrollo del módulo de recomendación: han proporcionado orientación sobre requisitos y realizaron consultas periódicas. Su implicación ha orientado decisiones de diseño y ha motivado la incorporación del módulo de electromovilidad y del sistema de predicción de precios, ampliando así la funcionalidad y el alcance de TankGo.

5.7 REPLANIFICACIONES REALIZADAS

A lo largo del proyecto se han producido varias replanificaciones que modificaron el alcance inicial. Estas desviaciones no respondieron a fallos de planificación, sino a la naturaleza exploratoria del proyecto y a oportunidades de colaboración que surgieron durante el desarrollo.

Ampliación del dominio de datos. El sistema ha sido concebido inicialmente como una plataforma de consulta de precios de gasolineras en tiempo real. A raíz de la colaboración con NextEdge y de la identificación de casos de uso más completos, el alcance se amplió para incluir puntos de recarga de vehículos eléctricos. Este cambio implicó la creación de un nuevo módulo y la revisión del modelo de datos para acomodar entidades heterogéneas.

Incorporación de datos históricos y predicción. El planteamiento original no contemplaba el almacenamiento de series históricas ni la predicción de precios. La disponibilidad de datos suficientes y el interés por ofrecer una propuesta de mayor valor añadido motivaron la incorporación del módulo de predicción basado en LightGBM, lo que requirió rediseñar la capa de persistencia y añadir una nueva tarea de entrenamiento y evaluación del modelo.

Adición del asistente conversacional. El agente de voz no figuraba en la planificación de la segunda fase. Su incorporación ha respondido al interés por explorar las posibilidades de interacción en lenguaje natural con el sistema, en línea con las tendencias actuales en interfaces conversacionales para aplicaciones de información geolocalizada.

Cambio de plataforma de despliegue. El primer despliegue se realizó en Render como solución de validación

rápida. La migración posterior a Google Cloud Platform supuso un esfuerzo de reconfiguración no previsto inicialmente, pero proporcionó una infraestructura más robusta, escalable y alineada con los objetivos de cloud-native del proyecto.

En conjunto, estas replanificaciones han resultado en un incremento del alcance respecto al prototipo inicial, sin comprometer los objetivos académicos ni los plazos de entrega del PFG.

6. CAPÍTULO SEXTO - PRESUPUESTO Y EVALUACIÓN DE COSTES

Este capítulo presenta la evaluación económica del proyecto TankGo desde dos perspectivas complementarias: el **coste teórico**, equivalente a contratar en el mercado el trabajo técnico realizado, y el **coste real** incurrido durante el desarrollo, que se ha mantenido mínimo gracias al uso estratégico de herramientas gratuitas y *free tiers* de los principales proveedores cloud. La diferencia entre ambas magnitudes ilustra que es posible construir y desplegar una plataforma de microservicios cloud-native de complejidad real con una inversión económica muy reducida.

6.1 COSTES LABORALES

El coste laboral es la partida más significativa del proyecto. El autor ha desempeñado simultáneamente varios roles técnicos a lo largo de los nueve meses de desarrollo, cuyas horas se distribuyen en la Tabla 6.1. Se emplea una tarifa de referencia de **30 €/hora**, correspondiente al perfil de Ingeniero de Software Junior con conocimientos en desarrollo fullstack, arquitecturas cloud y aprendizaje automático, coherente con las tablas salariales del mercado español en 2025–2026 [?].

Cuadro 6.1: Distribución de horas por rol y coste laboral estimado

Rol	Horas	Tarifa (€/h)	Subtotal (€)
Arquitecto de software / diseño del sistema	80	30	2.400
Desarrollador backend (Python / FastAPI)	120	30	3.600
Desarrollador backend (Node.js / Fastify/Hono)	80	30	2.400
Desarrollador frontend (React / TypeScript)	70	30	2.100
Ingeniería de datos / ML (LightGBM, pipeline)	60	30	1.800
DevOps / cloud / CI-CD (GCP, Docker)	50	30	1.500
Análisis de requisitos y documentación	40	30	1.200
Total estudiante	500	30	15.000

Conforme a la normativa del PFG, se incluye también el coste del director, estimado en 1 hora semanal de seguimiento durante 36 semanas más 10 horas de revisión de la memoria, a una tarifa de referencia de 60 €/hora para el perfil de Profesor Titular.

Cuadro 6.2: Coste del director del proyecto

Concepto	Horas	Tarifa (€/h)	Subtotal (€)
Reuniones y seguimiento semanal	36	60	2.160
Revisión de la memoria	10	60	600
Total director	46	60	2.760

6.2 MATERIAL, SOFTWARE E INFRAESTRUCTURA

El único elemento hardware empleado ha sido el ordenador personal del autor, valorado en 1.200 €. Aplicando amortización lineal a 4 años con un uso dedicado al proyecto del 60 % durante 9 meses, la amortización imputable es de **135 €**.

La práctica totalidad del software es de código abierto o gratuito: Python, Node.js, React, Docker Desktop, Visual Studio Code, Git/GitHub y Overleaf suponen un coste de **0 €**.

Respecto a la infraestructura cloud, todos los servicios utilizados durante el desarrollo (Cloud Run, Cloud Storage, Cloud Build, Artifact Registry, Neon PostgreSQL y Google AI Studio) se han mantenido dentro de sus niveles gratuitos. El único gasto efectivo ha sido el registro del dominio `tankgo.dev` por $\approx$ **11 €**.

6.3 COSTE TOTAL ESTIMADO DEL PROYECTO

La Tabla 6.3 consolida todas las partidas para obtener el coste total estimado si el proyecto se hubiera contratado en el mercado bajo condiciones laborales estándar.

Cuadro 6.3: Coste total estimado del proyecto

Partida	Importe (€)
Costes laborales — estudiante (500 h × 30 €/h)	15.000
Costes laborales — director (46 h × 60 €/h)	2.760
Amortización de hardware	135
Software y herramientas	0
Infraestructura cloud (dominio incluido)	11
Total estimado	17.906

El coste estimado de **17.906 €** refleja fielmente la complejidad del trabajo realizado: diseño de una arquitectura de microservicios con siete componentes, desarrollo fullstack, integración de modelos de aprendizaje automático, despliegue en Google Cloud Platform y extensión con capacidades conversacionales basadas en IA generativa.

6.4 ESTIMACIÓN DEL COSTE EN PRODUCCIÓN

Con el fin de contextualizar la viabilidad económica más allá de la fase académica, la Tabla 6.4 recoge una estimación orientativa del coste mensual de infraestructura para un escenario de producción real con carga moderada (500.000 peticiones mensuales).

Cuadro 6.4: Estimación orientativa de costes mensuales en producción

Servicio	Criterio de estimación	Coste mensual
Google Cloud Run (7 servicios)	Basado en la calculadora oficial de GCP [?] para 500.000 peticiones/mes con latencia media de 300 ms por servicio.	20 – 50 €
Neon PostgreSQL (plan Pro)	Plan mínimo de pago: 10 GiB de almacenamiento y cómputo escalable, suficiente para el volumen de datos de TankGo.	≈ 18 €
Google Cloud Storage	Almacenamiento de snapshots Parquet y modelos LightGBM.	< 1 €
Gemini 2.5 Flash [?]	Pipeline STT-LLM-TTS; coste unitario por interacción < 0,001 USD.	1 – 5 €
Dominio y DNS	Renovación anual de tankgo.dev.	≈ 1 €
Total mensual estimado		40 – 75 €/mes

TankGo podría operar como servicio real con un coste de infraestructura inferior a **1.000 €/año**, lo que refleja la eficiencia económica de las arquitecturas serverless y los modelos *pay-as-you-go* para proyectos de escala media, y confirma la viabilidad económica de la solución más allá del ámbito académico.

7. CAPITULO SEPTIMO

8. CAPÍTULO OCTAVO - DESARROLLO DEL SISTEMA

Este capítulo recoge la especificación, el diseño y la implementación de TankGo, así como el plan de pruebas y el manual de usuario. Es el núcleo técnico del proyecto y documenta las decisiones de arquitectura, las tecnologías empleadas y el resultado final del sistema.

8.1 ESPECIFICACIÓN DE REQUISITOS

8.1.1 Contexto del sistema

TankGo es una plataforma de inteligencia de datos orientada a la movilidad y la eficiencia energética. Su propósito es transformar los datos abiertos del Geoportal de Gasolineras del Ministerio para la Transición Ecológica [5] en información accionable para el conductor, añadiendo capas de valor mediante análisis histórico, predicción de precios y recomendación geoespacial de repostaje.

El sistema actúa como intermediario entre las fuentes de datos públicos y el usuario final, operando en un entorno cloud que permite tanto la consulta en tiempo real como el entrenamiento y servicio de modelos de machine learning. A diferencia de un visor estático, TankGo integra en una única plataforma la consulta de gasolineras y electrolineras, la estimación de precios futuros y un asistente conversacional por voz para interacción en manos libres.

8.1.2 Identificación de usuarios

El sistema contempla dos perfiles de usuario diferenciados. El **usuario no registrado** puede acceder a las funciones de consulta pública: visualización de gasolineras y electrolineras en mapa y tabla, detalle de cada estación y precios actuales. El **usuario registrado** dispone, además, de la gestión de estaciones favoritas, el acceso al histórico de precios de cada estación y las predicciones a 7 días generadas por el modelo de machine learning.

Dentro de estos perfiles, los casos de uso más representativos son el conductor particular que busca repostar barato en su área, el transportista profesional que planifica rutas de larga distancia con criterios de coste, y el usuario de vehículo eléctrico que necesita localizar puntos de recarga en ruta.

8.1.3 Requisitos funcionales

Los requisitos funcionales definen las capacidades que el sistema debe ofrecer al usuario [53]. La tabla 8.1 recoge los requisitos identificados para TankGo.

Cuadro 8.1: Requisitos funcionales del sistema

ID	Requisito	Descripción
RF-01	Consulta de gasolineras	El sistema muestra las estaciones de servicio cercanas a la ubicación del usuario en tabla y mapa interactivo, con detalle de precios por tipo de combustible.
RF-02	Consulta de electrolineras	El sistema muestra los puntos de recarga eléctrica disponibles en un mapa independiente.
RF-03	Histórico de precios	Para cada estación, el sistema permite visualizar la evolución histórica del precio de cada combustible.
RF-04	Predicción de precios	El sistema estima el precio de combustible para los próximos 7 días en las estaciones marcadas como favoritas por el usuario.
RF-05	Gestión de favoritos	El usuario registrado puede marcar y gestionar sus estaciones de servicio favoritas.
RF-06	Recomendación en ruta	El sistema sugiere paradas de repostaje óptimas a lo largo de una ruta, ponderando precio y desvío.
RF-07	Asistente conversacional	El sistema permite realizar consultas en lenguaje natural mediante voz, como localizar la gasolinera más barata en la zona.
RF-08	Autenticación de usuarios	El sistema permite el registro e inicio de sesión mediante Google OAuth.

8.1.4 Requisitos no funcionales

Los requisitos no funcionales establecen las propiedades de calidad del sistema, no directamente relacionadas con las funciones que ofrece, sino con cómo las ofrece [53].

Disponibilidad (RNF-01): Al tratarse de una plataforma de consulta en ruta, el sistema debe garantizar un tiempo de actividad superior al 99 %, respaldado por la infraestructura de Google Cloud Platform.

Latencia (RNF-02): El tiempo de respuesta de las consultas de precios y el asistente conversacional no deben ser muy elevados, requisito crítico para garantizar la seguridad vial en uso móvil.

Escalabilidad (RNF-03): La arquitectura de microservicios y el despliegue en Cloud Run permiten escalar cada componente de forma independiente según la demanda, sin afectar al resto del sistema.

Usabilidad (RNF-04): El frontend está diseñado con criterio *mobile-first* y es completamente responsivo, priorizando la experiencia en dispositivos móviles sin renunciar a la usabilidad en escritorio.

Seguridad (RNF-05): Las comunicaciones se realizan sobre HTTPS, la autenticación se gestiona mediante tokens JWT y el acceso a los microservicios internos está restringido a través del API Gateway.

8.1.5 Restricciones

El sistema opera bajo tres restricciones principales. En primer lugar, existe una **dependencia de fuentes externas**: los datos de gasolineras provienen de la API pública del Ministerio para la Transición Ecológica [4], cuya disponibilidad y frecuencia de actualización están fuera del control del sistema. En segundo lugar, las funciones de consulta, predicción y recomendación requieren **conectividad activa a internet**, ya que toda la lógica reside en el backend cloud. Finalmente, el **asistente conversacional** está diseñado para responder consultas puntuales, no para mantener diálogos extendidos mientras se conduce, en línea con las directrices de seguridad vial aplicables. El sistema de recomendación de rutas también añade una dependencia esta fuera del control del sistema, ya que se basa en la API de Open Route Service para el cálculo de rutas y distancias.

8.2 DISEÑO DEL SISTEMA

8.2.1 Alternativas exploradas y decisiones tecnológicas

Las decisiones tecnológicas de TankGo responden a criterios de adecuación funcional, rendimiento en entornos cloud y familiaridad del equipo de desarrollo. A continuación se justifican las elecciones más relevantes.

Para el **API Gateway** se han evaluado opciones como Kong, AWS API Gateway y soluciones basadas en Nginx. Se ha optado por **Hono sobre Node.js** por su bajo consumo de recursos, su rendimiento en entornos serverless y su sintaxis próxima a Express, lo que simplificó la curva de aprendizaje [29]. Hono permite implementar enrutamiento, middleware de autenticación y políticas CORS con un código mínimo y muy legible.

Para los **microservicios de dominio** se han utilizado dos tecnologías según el lenguaje más adecuado para cada caso. El servicio de usuarios se implementó con **Fastify (Node.js)**, un framework orientado al alto rendimiento con una gestión eficiente del esquema de validación. Los servicios de gasolineras, recomendación y predicción se implementaron con **FastAPI (Python)**, elección motivada por la integración nativa con el ecosistema de machine learning de Python (scikit-learn, LightGBM, pandas) y su capacidad para generar documentación OpenAPI automáticamente.

Para el **frontend** se ha optado por **React con TypeScript y Vite**, combinación que ofrece tipado estático, tiempos de compilación reducidos y un ecosistema de componentes amplio. La aplicación está construida como *Progressive Web App* (PWA), lo que permite su uso en dispositivos móviles con comportamiento nativo.

La **base de datos** es PostgreSQL, desplegada en Neon Cloud [?]. Se ha elegido PostgreSQL por su solidez, soporte de extensiones geoespaciales y compatibilidad con los ORM utilizados en ambos stacks tecnológicos.

8.2.2 Arquitectura general

TankGo sigue una **arquitectura de microservicios** en la que cada funcionalidad principal del sistema se encapsula en un servicio independiente con su propia lógica y despliegue. Los servicios se comunican entre sí a través de una capa de API Gateway que actúa como punto de entrada único para el cliente.

La arquitectura se organiza en cuatro niveles. El **cliente** es la aplicación React PWA que el usuario final consume desde el navegador o dispositivo móvil. El **API Gateway** (Hono) recibe todas las peticiones del cliente, gestiona la autenticación y las enruta hacia el microservicio correspondiente. La **capa de servicios** agrupa los cinco microservicios del dominio: usuarios, gasolineras, electrolinerías, recomendación y predicción. Finalmente, la **capa de persistencia** centraliza el almacenamiento en PostgreSQL, accesible desde cada servicio según sus necesidades.

Figura 8.1: Diagrama de arquitectura general de TankGo

8.2.3 Diseño lógico y físico

Desde el punto de vista **lógico**, el sistema se estructura en torno a cinco entidades de dominio principales: *Estación de servicio*, *Precio*, *Usuario*, *Favorito* y *Predicción*. Las estaciones concentran la información geolocalizada de cada gasolinera o electrolinería; los precios registran tanto el valor actual como el histórico asociado a cada

estación y tipo de combustible; los usuarios encapsulan la identidad y preferencias de cada cuenta; los favoritos modelan la relación entre usuario y estación; y las predicciones almacenan las estimaciones generadas por el modelo de machine learning para su consulta posterior.

Desde el punto de vista **físico**, cada microservicio se despliega como un contenedor Docker independiente. En producción, los servicios corren en **Google Cloud Run**, que gestiona el escalado automático en función de la demanda. La base de datos reside en **Neon Tech** (PostgreSQL), y los artefactos de los modelos de machine learning se almacenan en **Cloud Storage**. El tráfico externo entra exclusivamente por el API Gateway, que es el único componente con URL pública expuesta.

8.2.4 Modelo de datos

El esquema de base de datos de TankGo se compone de cinco tablas principales cuyo diseño refleja directamente el dominio del sistema.

La tabla `gasolineras` es la entidad central del modelo. Almacena la información estática y los precios actuales de cada estación de servicio, identificada de forma única mediante el campo `ideess` (identificador oficial del Ministerio). Incluye datos de localización (municipio, provincia, dirección, coordenadas y geometría PostGIS), horario en formato estructurado (`horario_parsed` en JSONB) y los precios actuales de cada tipo de combustible: Gasolina 95 E5, 98 E5, Gasóleo A, Gasóleo B, Gasóleo Premium, Gasolina 95 Premium y Diésel Renovable.

La tabla `precios_historicos` registra la evolución temporal de los precios por estación. Cada registro asocia un `ideess` con una fecha y los precios de los distintos combustibles en ese momento, lo que permite construir series temporales para el análisis histórico y el entrenamiento del modelo de predicción.

La tabla `prediction_forecasts` almacena las estimaciones generadas por el modelo LightGBM. Su clave primaria es compuesta: `ideess`, `fuel`, `forecast_date` y `run_date`, lo que permite distinguir predicciones para diferentes combustibles, horizontes temporales y ejecuciones del modelo. Cada registro incluye el precio predicho y los márgenes de confianza inferior y superior.

La tabla `users` gestiona las cuentas de usuario, incluyendo autenticación tradicional mediante `password_hash` y autenticación federada mediante `google_id` (OAuth). Almacena también las preferencias del usuario como el combustible favorito, el modelo de coche y el tipo de combustible del vehículo.

La tabla `user_favorites` modela la relación entre usuarios y estaciones, permitiendo que cada usuario registre sus gasolineras favoritas. Referencia mediante claves foráneas tanto a `users.id` como a `gasolineras.ideess`.

Las dependencias entre tablas son las siguientes: `precios_historicos` y `user_favorites` referencian `gasolineras` a través de `ideess`; `prediction_forecasts` referencia igualmente a `gasolineras`; y `user_favorites` referencia a `users` mediante `user_id`. El diagrama entidad-relación de la figura 8.2 ilustra estas relaciones.

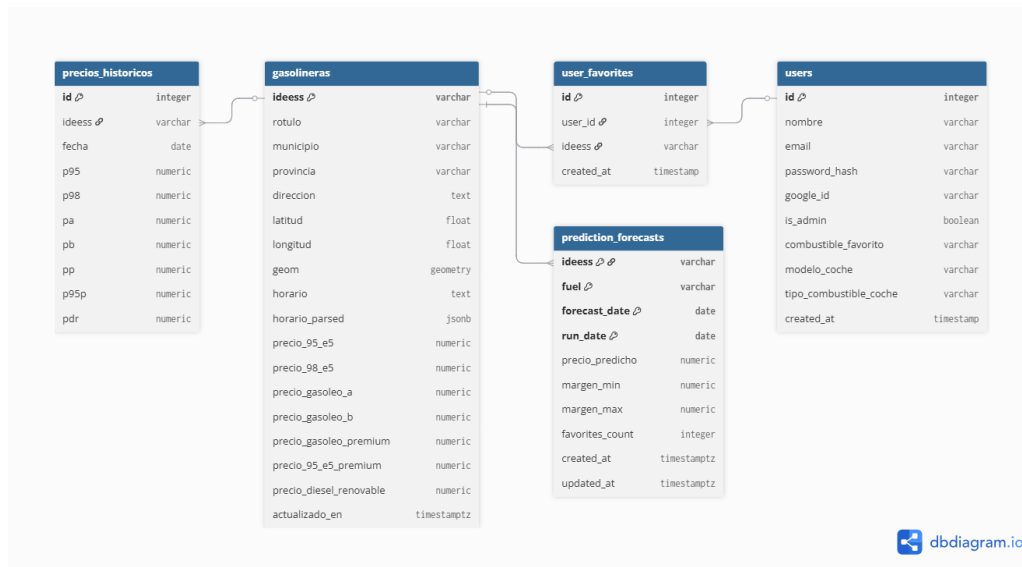


Figura 8.2: Modelo entidad-relación del sistema

8.2.5 Diagramas UML

El diagrama de casos de uso recoge las interacciones entre los actores del sistema y sus funcionalidades. El *usuario no registrado* puede visualizar gasolineras en mapa y tabla, consultar el mapa de electrolineras y registrarse o iniciar sesión. El *usuario registrado* extiende esas capacidades con acceso a la tabla de electrolineras, la planificación de rutas con recomendación de repostaje, el asistente conversacional, la predicción de precios a 7 días, la gestión de favoritos y el histórico de precios. El sistema interactúa además con tres actores externos: la API del Ministerio (MITECO) para la sincronización de precios, la Reve API para los puntos de recarga eléctrica, y OpenRouteService para el cálculo de rutas.

Figura 8.3: Diagrama de casos de uso del sistema

El diagrama de secuencia de la figura 8.4 ilustra el flujo de consulta de gasolineras cercanas: el usuario activa la geolocalización, el frontend envía la petición al API Gateway, este la enruta al servicio de gasolineras, que

ejecuta una consulta geoespacial sobre PostgreSQL mediante PostGIS y devuelve el listado de estaciones con sus precios actuales para su renderizado en tabla y mapa.

Figura 8.4: Diagrama de secuencia: consulta de gasolineras cercanas

El segundo diagrama de secuencia, figura 8.5, muestra el flujo de planificación de ruta con recomendación de repostaje. El servicio de recomendación coordina dos llamadas externas: a OpenRouteService para obtener el trazado de la ruta, y al servicio de gasolineras para recuperar las estaciones cercanas al recorrido. Con ambas respuestas aplica el algoritmo de ponderación por precio y desvío y devuelve las paradas sugeridas al usuario.

Figura 8.5: Diagrama de secuencia: planificación de ruta con recomendación de repostaje

8.3 CONSIDERACIONES SOBRE LA IMPLEMENTACIÓN

Esta sección documenta las decisiones de implementación más relevantes de cada componente del sistema, con especial atención a los aspectos que condicionan el comportamiento en producción y que no quedan completamente cubiertos por la especificación de requisitos o el diseño arquitectónico.

8.3.1 Estructura del proyecto

El repositorio sigue una estructura de monorepo, cada microservicio reside en su propio directorio raíz y tiene su propio `Dockerfile`, su fichero de dependencias y su conjunto de pruebas. Esta organización permite iterar sobre un servicio sin alterar el resto y facilita la asignación de un pipeline de CI/CD independiente a cada componente.

Los directorios principales son `gateway-hono`, `gasolineras-service`, `usuarios-service`, `recomendacion-service`, `prediction-service`, `voice-assistant-service`, `mcp-gasolineras-server` y `frontend-client`. En la raíz se encuentran el `docker-compose.yml` para el entorno local y los siete ficheros `cloudbuild-*.yaml` para el despliegue en producción.

La orquestación local se basa en **Docker Compose** [54], con dependencias de arranque declaradas explícitamente, los servicios de dominio esperan a que la base de datos esté accesible, el gateway espera a que los servicios de dominio superen sus *health checks*, el frontend arranca únicamente cuando el gateway está dis-

ponible. Los servicios opcionales de IA y predicción se activan mediante perfiles Docker (ai, ml) [55], de forma que la instalación base no requiere claves de API externas.

8.3.2 Servicio de gasolineras y sincronización de datos

El `gasolineras-service` es el componente con mayor complejidad operacional del sistema, ya que combina la gestión de datos en tiempo real con la necesidad de mantener fresca respecto a una fuente externa que actualiza sus precios varias veces al día [4].

Ciclo de sincronización

El servicio implementa tres mecanismos de sincronización complementarios. En primer lugar, al arrancar, si la variable `AUTO_ENSURE_FRESH_ON_STARTUP` está activa, el servicio comprueba la antigüedad de los datos almacenados y lanza una sincronización condicional si supera el umbral configurado. En segundo lugar, en producción un **Cloud Scheduler** de Google Cloud Platform [56, 57] dispara diariamente un Cloud Run Job [13] que invoca el endpoint interno `POST /gasolineras/daily-sync-export`, este endpoint sincroniza los datos desde la API del Ministerio de Industria y, a continuación, exporta un snapshot en formato Parquet a Google Cloud Storage [47] para el pipeline de machine learning. En tercer lugar, si una petición de lectura detecta que los datos superan el umbral de antigüedad (`AUTO_SYNC_ON_READ=true`), el servicio puede desencadenar una sincronización en segundo plano, aunque en producción esta opción se mantiene desactivada para evitar latencia impredecible.

La combinación de estos mecanismos garantiza que los datos nunca superen 24 horas de antigüedad en producción sin necesidad de un sistema de colas externo.

Almacenamiento y consultas geoespaciales

Los precios actuales se almacenan en la tabla `gasolineras` junto con las coordenadas geográficas de cada estación. Para las búsquedas por proximidad se utiliza **PostGIS** [58], la extensión geoespacial de PostgreSQL, que permite ejecutar consultas basadas en distancia con los operadores `ST_DWithin` y `ST_Distance` de forma eficiente mediante índices espaciales. Esto evita el cálculo de la distancia haversine en capa de aplicación para el filtrado inicial, reservándolo únicamente para el refinamiento de resultados.

El histórico de precios se registra en la tabla `precios_historicos`, donde cada fila representa una instantánea de los precios de una estación en una fecha concreta. Este diseño permite construir series temporales por

estación y combustible, que son la entrada del modelo de predicción.

Fallback en memoria

Si al arrancar el servicio no dispone de conexión a la base de datos, entra en modo *memory-fallback*: carga los datos desde la última sincronización disponible en memoria y continúa respondiendo peticiones, aunque sin capacidad de escritura. Este modo garantiza la resiliencia ante interrupciones transitorias de conectividad con Neon, que al ser un servicio externo puede experimentar latencias de conexión en arranques en frío (*cold starts*) de Cloud Run.

Integración de electrolineras

Los puntos de recarga eléctrica se obtienen de la API de Mapareve [9] y están integrados directamente dentro del `gasolineras-service`, bajo el prefijo de rutas `/api/charging/`. Esta decisión evita la proliferación de microservicios para funcionalidades que comparten el mismo patrón de acceso (consulta geoespacial, caché corta) y reduce la complejidad operacional del despliegue.

8.3.3 Servicio de usuarios y autenticación

El `usuarios-service` gestiona el ciclo completo de identidad: registro, autenticación tradicional, autenticación federada con Google OAuth y gestión de favoritos. Está implementado con **Fastify** [59], que en combinación con el plugin `@fastify/jwt` [60] proporciona la firma y verificación de tokens sin código boilerplate.

Autenticación y tokens JWT

Tras un inicio de sesión correcto, el servicio genera un token **JWT** [?] firmado con el algoritmo HS256 y una caducidad de 7 días. El token no se devuelve en el cuerpo de la respuesta, sino que el API Gateway lo establece como una *cookie* `httpOnly` con los flags `Secure` y `SameSite=None` en producción. Este mecanismo impide que el token sea accesible desde JavaScript del navegador, mitigando ataques XSS [61].

Para el flujo de Google OAuth, el gateway actúa como intermediario: recibe el `id_token` de Google, lo verifica y llama al endpoint interno `POST /api/usuarios/google/internal` con cabecera `X-Internal-Secret`, de forma que este endpoint nunca queda expuesto al exterior. El servicio realiza un *upsert* del usuario (crea la cuenta si no existe, o la actualiza si ya existe por email) y devuelve el JWT al gateway para su encapsulación en cookie.

Las contraseñas se almacenan únicamente como hash generado con **bcrypt** [?] con 12 rondas de sal, lo que hace computacionalmente inviable su recuperación por fuerza bruta incluso en caso de exfiltración de la base de datos.

Gestión del esquema de base de datos

El proyecto no utiliza un sistema de migraciones automático tipo Flyway o Liquibase. El esquema se define en un fichero `init.sql` con sentencias idempotentes (`CREATE TABLE IF NOT EXISTS`, `ALTER TABLE IF NOT EXISTS`) que se aplican manualmente sobre la instancia PostgreSQL de Neon en el primer despliegue. Esta decisión es apropiada para el alcance del proyecto, aunque en un sistema de producción con múltiples entornos sería recomendable introducir un gestor de migraciones formal para controlar la evolución del esquema a lo largo del tiempo.

Rate limiting

Los endpoints de registro e inicio de sesión aplican un límite de 5 peticiones por IP cada 15 minutos mediante el plugin `@fastify/rate-limit` [62], configurado con `global: false` para no afectar al resto de rutas. Las direcciones de loopback (`127.0.0.1`, `::1`) quedan exentas para no interferir con las pruebas locales.

8.3.4 Servicio de recomendación de ruta

8.3.5 Módulo de predicción de precios

El `prediction-service` implementa un pipeline de machine learning para estimar el precio de combustible de las estaciones favoritas de los usuarios durante los próximos 7 días. El diseño se orienta a la ejecución periódica en lote (*batch*) más que a la inferencia en tiempo real.

Selección del modelo

Se evaluaron varios enfoques de regresión para series temporales: modelos ARIMA clásicos, redes LSTM y árboles potenciados por gradiente. Se ha optado por **LightGBM** [?] en régimen de **regresión cuantílica** por tres razones principales. Primero, ofrece un rendimiento muy competitivo en tablas de series temporales con features derivadas sin requerir la normalización y el ajuste de hiperparámetros extensivo que exigen las redes neuronales recurrentes. Segundo, la regresión cuantílica (cuantiles 0.05, 0.50, 0.95) produce directamente intervalos

de confianza para cada predicción, lo que enriquece la información presentada al usuario. Tercero, LightGBM tiene un tiempo de entrenamiento muy reducido, compatible con la ejecución semanal del job en un entorno serverless con límites de memoria y CPU [?].

Figura 8.6: Comparativa de MAE entre modelos evaluados durante el desarrollo

Ingeniería de *features*

Para cada estación y tipo de combustible se construye una serie temporal diaria con el precio medio registrado. A partir de ella se generan *features* temporales (día de la semana, fin de semana, semana del año, mes, trimestre), *lags* de 1, 7, 14, 21 y 30 días, medias móviles y desviaciones típicas de ventanas de 7, 14 y 30 días, y diferencias a 1 y 7 días. Adicionalmente, si la conexión a internet está disponible, se descarga el precio del crudo Brent mediante **yfinance** [?] y se incorporan sus *lags* como variable exógena, aprovechando la correlación conocida entre el precio del petróleo y los precios en surtidor [7].

Ciclo de entrenamiento e inferencia

El pipeline se ejecuta semanalmente mediante un **Cloud Run Job** disparado por **Cloud Scheduler** (cada lunes a las 06:00 UTC). En cada ejecución, el job descarga los snapshots Parquet de los últimos 180 días desde Google Cloud Storage, valida que los datos estén frescos, reentrena el modelo para cada par (estación, combustible), genera las predicciones de los 7 días siguientes de forma secuencial —cada día nuevo utiliza las predicciones previas para recalcular los *lags*— y persiste los resultados en la tabla `prediction_forecasts` de PostgreSQL. Los artefactos del modelo entrenado (.pkl) se almacenan en el bucket GCS bajo el prefijo `modelos_lgbm/` para su eventual reutilización.

La selección de estaciones a predecir se basa en los favoritos globales de los usuarios: el job consulta el endpoint interno `/api/usuarios/favoritos/all-ideess` para obtener los identificadores únicos de las estaciones con al menos un usuario que las ha marcado como favoritas. Este criterio concentra la capacidad de cómputo en las estaciones con mayor demanda real, evitando el coste prohibitivo de entrenar modelos para las más de 11 000 estaciones del catálogo nacional.

Consulta de predicciones

En producción el servicio opera en modo API (`RUN_HTTP_API=true`): no ejecuta el pipeline de entrenamiento, sino que expone el endpoint `GET /api/prediction/station/{ideess}` que lee directamente de la tabla `prediction_forecasts`. El entrenamiento y la escritura en base de datos son responsabilidad exclusiva del Cloud Run Job periódico, separando claramente los roles de escritura y lectura.

8.3.6 Asistente conversacional de voz

El `voice-assistant-service` implementa un asistente de dominio restringido que permite al usuario realizar consultas sobre gasolineras en lenguaje natural, tanto por texto como por voz, y recibir una respuesta en audio. El servicio está implementado con **Fastify** [59] y se integra con la API de **Google Gemini** [?] mediante llamadas HTTP REST al cliente `@google/genai`.

Pipeline STT → LLM → TTS

El pipeline se ejecuta en tres etapas secuenciales, cada una con su propia llamada a la API de Gemini. En la etapa de **reconocimiento de voz** (*Speech-to-Text*), si la petición incluye audio en base64, el servicio invoca `generateContent` con el modelo configurado para transcribir el audio a texto. En la etapa de **razonamiento** (*LLM con function calling*), se realiza una primera llamada con las declaraciones de las funciones disponibles (`get_prices` y `get_nearest_stations`); si el modelo solicita ejecutar alguna de ellas, el servicio las invoca localmente contra el API Gateway y realiza una segunda llamada con los resultados, obteniendo así la respuesta final fundamentada en datos reales. En la etapa de **síntesis de voz** (*Text-to-Speech*), se realiza una llamada adicional a Gemini con modalidad de respuesta `AUDIO`, la respuesta PCM se convierte a WAV y se devuelve en base64 al cliente.

Este diseño con tres llamadas separadas, en lugar de una única llamada multimodal, ofrece mayor control sobre cada etapa y facilita el manejo de errores y los fallbacks por modelo.

Guardarraíles y control del dominio

Dado que el asistente está diseñado para un dominio muy específico, se implementan varios mecanismos para evitar un uso fuera de ámbito. Un **comprobador de alcance** (*scope guard*) basado en palabras clave evalúa la transcripción antes de invocar el LLM; si la consulta no contiene términos relacionados con combustible, gasolineras, precios o navegación, y la variable `VOICE_GUARDRAILS_ENABLED` está activa, el servicio devuelve

una respuesta predefinida sin consumir tokens del modelo. El modo `strict` rechaza directamente la petición; el modo `soft` permite continuar con una advertencia al modelo.

Adicionalmente, el **system prompt** construido por `buildSystemInstruction` instruye explícitamente al modelo sobre el idioma, el tono y el formato esperados, y lo obliga a utilizar las funciones declaradas para cualquier consulta de datos, reduciendo el riesgo de alucinaciones. El function calling se ejecuta en el servidor, de forma que el LLM nunca interactúa directamente con el gateway, sino que proporciona los parámetros de la llamada y espera los resultados.

Otros controles operativos incluyen límites de tamaño de payload (`VOICE_MAX_TEXT_CHARS`, `VOICE_MAX_AUDIO_BASE64_CHARS`), timeouts por llamada a Gemini con `AbortController`, y rate limiting por IP para el transporte WebSocket. Como limitación conocida, el rate limiter no se aplica actualmente al endpoint HTTP `/voice/dialog`, aspecto identificado para una iteración futura.

Nota sobre el transporte WebSocket

El servicio implementa tanto el endpoint HTTP `POST /voice/dialog` como un endpoint WebSocket `/ws/voice` para streaming de audio. Durante el desarrollo se identificaron problemas de estabilidad de conexión con el transporte WebSocket en el entorno de Cloud Run, asociados al comportamiento de las peticiones de larga duración en infraestructura serverless. Por ello, el cliente frontend utiliza actualmente el endpoint HTTP, que ofrece un comportamiento predecible en todos los entornos de despliegue.

8.3.7 Frontend: React PWA con Nginx

El frontend es una *Single Page Application* (SPA) construida con **React 19** [16] y **TypeScript** [?], compilada con **Vite** [?] y empaquetada como **Progressive Web App** (PWA) [?]. En producción, los artefactos estáticos generados por Vite se sirven mediante **Nginx** [?] dentro de un contenedor Docker de imagen mínima.

Configuración de Nginx

Nginx está configurado para servir el directorio `dist` generado por Vite desde `/usr/share/nginx/html`. La directiva `try_files $uri $uri/ /index.html` garantiza que todas las rutas no estáticas sean resueltas por el router de React en cliente, comportamiento necesario para el correcto funcionamiento de la navegación SPA. Los assets estáticos con huella de contenido (`.js`, `.css`, fuentes e imágenes) se cachean durante un año con la cabecera `Cache-Control: public, immutable`, lo que maximiza el rendimiento en visitas recurrentes.

Nginx no actúa como proxy inverso para la API en producción: todas las llamadas al backend se dirigen directamente a la URL del API Gateway configurada en tiempo de compilación mediante la variable `VITE_API_BASE_URL`.

Progressive Web App

La aplicación implementa el estándar PWA mediante el plugin **vite-plugin-pwa** [?] con Workbox [26] como gestor del Service Worker. El fichero `manifest.webmanifest` define los metadatos necesarios para la instalación en dispositivos móviles (nombre, iconos, color de tema y modo de visualización *standalone*). El Service Worker implementa una estrategia de caché que permite el acceso a la interfaz aunque la conexión sea intermitente, aunque las consultas de datos en tiempo real requieren conectividad activa.

Internacionalización y diseño responsivo

La interfaz soporta tres idiomas (español, inglés y euskera) mediante **i18next** [27] con detección automática del idioma del navegador. El diseño sigue un enfoque *mobile-first* con **TailwindCSS** [?], garantizando que la experiencia sea funcional tanto en pantallas de escritorio como en dispositivos móviles, donde el caso de uso de búsqueda de gasolineras en ruta es predominante. El mapa interactivo se implementa con **Leaflet** [?] a través de la librería `react-leaflet`, con soporte para clustering de marcadores a niveles de zoom bajos para evitar la saturación visual cuando hay muchas estaciones en el área visible.

8.3.8 Despliegue en GCP y pipeline CI/CD

Todos los microservicios se despliegan en **Google Cloud Run** [?], plataforma serverless que gestiona el escalado automático en función de la demanda y factura únicamente por el tiempo de cómputo efectivo. Esta elección elimina la necesidad de gestionar instancias o clústeres, lo que reduce significativamente la carga operacional para un proyecto académico sin comprometer la disponibilidad.

Pipeline de CI/CD

Cada microservicio dispone de su propio fichero `cloudbuild-*.yaml` que define un pipeline de **Cloud Build** [?] independiente. Los pipelines se activan automáticamente con cada *push* a la rama `main` del repositorio. El proceso consta de tres pasos: construcción de la imagen Docker, publicación en **Artifact Registry** [49] (región `europa-west1`), y despliegue en Cloud Run con las variables de entorno inyectadas desde **Secret Manager** [51]. Este diseño garantiza que ningún secreto esté presente en el repositorio ni en los logs de compilación.

El pipeline del frontend incluye validaciones adicionales en el paso de compilación: la variable `VITE_API_BASE_URL` debe apuntar a un dominio HTTPS (no a `localhost`), lo que previene despliegues accidentales de configuraciones de desarrollo.

Comunicación entre servicios en Cloud Run

En Cloud Run cada servicio tiene una URL pública asignada por Google, pero la comunicación interna entre servicios no debe transitar por internet. Para ello, el gateway utiliza tokens IAM de servicio (*service account tokens*) inyectados mediante el módulo `cloudRunAuthFetch.js`, que autentica cada llamada inter-servicio ante la infraestructura de Google sin exponer los servicios internos a peticiones externas no autorizadas.

Base de datos y almacenamiento

La base de datos PostgreSQL reside en **Neon Tech** [?], proveedor de PostgreSQL serverless que ofrece *branching* de bases de datos, hibernación automática de instancias inactivas y conexión mediante cadena estándar `postgresql://`. La elección de un proveedor externo en lugar de Cloud SQL de GCP responde a la disponibilidad de un plan gratuito suficiente para las cargas del proyecto y a la independencia del proveedor cloud para la capa de datos.

Los snapshots Parquet del pipeline de ML y los modelos entrenados se almacenan en **Google Cloud Storage** [?], que actúa como capa de almacenamiento de objetos entre el servicio de gasolineras (productor de datos), el job de predicción (consumidor y productor de modelos) y el servicio de predicción en modo API (lector de resultados).

8.3.9 Seguridad transversal

Además de los mecanismos de autenticación descritos en la sección anterior, el sistema implementa un conjunto de controles de seguridad aplicados de forma transversal.

La comunicación entre el gateway y los microservicios internos está protegida por la cabecera `X-Internal-Secret`: los endpoints de administración (sincronización forzada, export, consulta de favoritos globales) rechazan con `403 Forbidden` cualquier petición que no incluya el secreto configurado en `INTERNAL_API_SECRET`. Este mecanismo evita que dichos endpoints sean accesibles desde el exterior incluso si un servicio queda expuesto inadvertidamente.

La validación de datos de entrada se realiza en capa de aplicación con **Pydantic v2** [35] en los servicios Python y con los esquemas de validación de Fastify en los servicios Node.js. Los parámetros de consulta con potencial de abuso (radio de búsqueda, límite de resultados) tienen cotas máximas declaradas explícitamente. Las consultas a PostgreSQL utilizan siempre parámetros preparados, sin concatenación de cadenas, eliminando el vector de inyección SQL [?].

El servicio de usuarios aplica **Helmet.js** [?] para establecer cabeceras HTTP de seguridad (`X-Frame-Options`, `Content-Security-Policy`, `X-Content-Type-Options`). En producción, Cloud Run gestiona HTTPS automáticamente, de forma que todo el tráfico externo está cifrado en tránsito. Los secretos de producción (claves de API, cadenas de conexión, JWT secret) nunca residen en el repositorio y se inyectan en tiempo de ejecución desde Secret Manager.

8.4 PLAN DE PRUEBAS

8.4.1 Estrategia

8.4.2 Pruebas unitarias

8.4.3 Pruebas de integración

8.4.4 Pruebas funcionales

8.4.5 Resultados obtenidos

8.5 MANUAL DE USUARIO

8.5.1 Acceso al sistema

8.5.2 Uso de la plataforma

8.5.3 Capturas de pantalla

8.6 INCIDENCIAS DEL PROYECTO

BIBLIOGRAFÍA

- [1] I. Alvis, “TankGo: Plataforma Inteligente de Precios de Carburantes,” Universidad de Deusto, Proyecto académico de la asignatura: Desarrollo Avanzado de Aplicaciones para la Web de Datos, Diciembre 2025, tutor: Diego López de Ipiña González de Artaza.
- [2] P. Mell and T. Grance, “The nist definition of cloud computing,” NIST, Tech. Rep. SP 800-145, 2011. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>
- [3] EXPANSIÓN, “España supera los 600.000 coches eléctricos e híbridos enchufables,” Expansión (sección Motor), enero 2026, consultado el 5 de marzo de 2026. [Online]. Available: <https://www.expansion.com/empresas/motor/2026/01/12/69650923468aeb52078b4580.html>
- [4] datos.gob.es, “Precio de carburantes en las gasolineras españolas,” 2026, portal de datos abiertos del Gobierno de España. Consultado el 5 de marzo de 2026. [Online]. Available: <https://datos.gob.es/es/catalogo/e05068001-precio-de-carburantes-en-las-gasolineras-espanolas>
- [5] Ministerio para la Transición Ecológica y el Reto Demográfico, “Geoportal de gasolineras: precios de carburantes,” 2026, fuente oficial de precios. Consultado el 5 de marzo de 2026. [Online]. Available: <https://sedeaplicaciones.minetur.gob.es/ServiciosRESTCarburantes/PreciosCarburantes/EstacionesTerrestres/>
- [6] J. Guo, “Oil price forecast using deep learning and arima,” in *2019 International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI)*, 2019, pp. 241–247.
- [7] S. Lahmiri, “Fossil energy market price prediction by using machine learning with optimal hyper-parameters: A comparative study,” *Resources Policy*, vol. 92, p. 105008, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0301420724003751>
- [8] P. A. R Azmi, M. Yusoff, and M. T. Mohd Sallehud-din, “A review of predictive analytics models in the oil and gas industries,” *Sensors*, vol. 24, no. 12, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/12/4013>
- [9] Ministerio de Transportes y Movilidad Sostenible, “Estrategia de Movilidad Segura, Sostenible y Conectada 2030 (es.movilidad),” 2026, página oficial del ministerio sobre la estrategia es.movilidad. Consultado el 5 de marzo de 2026. [Online]. Available: <https://www.transportes.gob.es/ministerio/planes-estrategicos/esmovilidad>

- [10] OpenStreetMap contributors, “Routing/online routers — OpenStreetMap wiki,” 2025, consultado: mayo de 2025. [Online]. Available: https://wiki.openstreetmap.org/wiki/Routing/online_routers
- [11] Gartner Inc., “Top 10 Strategic Technology Trends for 2026,” Gartner, Tech. Rep., 2025, accedido en Marzo de 2026. [Online]. Available: <https://www.gartner.com/en/articles/top-technology-trends-2026>
- [12] Databricks, “What is Parquet? columnar file format vs CSV, Avro & ORC,” 2024, consultado: mayo de 2025. [Online]. Available: <https://www.databricks.com/glossary/what-is-parquet>
- [13] Google Cloud, “Cloud Run – fully managed compute platform,” 2026, consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/run/docs?hl=es>
- [14] United Nations, “Objetivos de Desarrollo Sostenible,” 2026, portal oficial de la ONU sobre los ODS. Consultado el 5 de marzo de 2026. [Online]. Available: <https://sdgs.un.org/es/goals>
- [15] Gobierno de España, “Agenda 2030 y Objetivos de Desarrollo Sostenible,” 2026, información institucional sobre la Agenda 2030 en España. Consultado el 5 de marzo de 2026. [Online]. Available: <https://www.mdsocialesa2030.gob.es/agenda2030/index.htm>
- [16] Meta (React), “React — A JavaScript library for building user interfaces,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://react.dev/>
- [17] Vue.js Team, “Vue.js — The Progressive JavaScript Framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://vuejs.org/>
- [18] Angular Team, “Angular — One framework. Mobile and desktop.” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://angular.dev/>
- [19] React-Leaflet Project, “React-Leaflet: React components for Leaflet maps,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://react-leaflet.js.org/>
- [20] Stack Overflow, “Stack Overflow Developer Survey 2025 – Technology,” 2025, consultado: mayo de 2026. [Online]. Available: <https://survey.stackoverflow.co/2025/technology/>
- [21] LogRocket Blog, “Leaflet adoption guide: Overview, examples, and alternatives,” 2024, consultado: mayo de 2025. [Online]. Available: <https://blog.logrocket.com/leaflet-adoption-guide/>
- [22] Geoapify, “Map libraries popularity: Leaflet vs MapLibre GL vs OpenLayers,”

- 2024, consultado: mayo de 2025. [Online]. Available: <https://www.geoapify.com/map-libraries-comparison-leaflet-vs-maplibre-gl-vs-openlayers-trends-and-statistics/>
- [23] Leaflet.markercluster contributors, “Leaflet.markercluster – marker clustering for leaflet,” 2023, repositorio y documentación. Consultado el 21 de mayo de 2026. [Online]. Available: <https://github.com/Leaflet/Leaflet.markercluster>
- [24] Google Developers, “Progressive web apps (pwa) – concepts and best practices,” 2024, guía de PWA en web.dev. Consultado el 21 de mayo de 2026. [Online]. Available: <https://web.dev/progressive-web-apps/>
- [25] MDN Web Docs, “Service workers – mozilla developer network,” 2025, referencia técnica sobre Service Workers. Consultado el 21 de mayo de 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- [26] Google Workbox Project, “Workbox – libraries for production-ready service workers,” 2024, guía y utilidades para Service Workers. Consultado el 21 de mayo de 2026. [Online]. Available: <https://developers.google.com/web/tools/workbox>
- [27] i18next contributors, “i18next – internationalization framework,” 2024, documentación oficial. Consultado el 21 de mayo de 2026. [Online]. Available: <https://www.i18next.com/>
- [28] react-i18next contributors, “react-i18next – internationalization for react,” 2024, adaptación de i18next para React. Consultado el 21 de mayo de 2026. [Online]. Available: <https://react.i18next.com/>
- [29] Hono Project, “Hono - ultrafast web framework for the edge,” 2024, accessed: 2025-03. [Online]. Available: <https://hono.dev>
- [30] —, “Hono — ultrafast web framework for the edges,” 2024, consultado: mayo de 2025. [Online]. Available: <https://hono.dev/docs/concepts/benchmarks>
- [31] Express.js, “Express - Fast, unopinionated, minimalist web framework for Node.js,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://expressjs.com/>
- [32] I. J. Afolabi, “API Framework Performance Benchmark: FastAPI vs Flask vs Django,” 2025, consultado: mayo de 2025. [Online]. Available: <https://medium.com/@afolabiifeoluwa06/api-framework-performance-benchmark-fastapi-vs-flask-vs-django-e767ad51574c>

- [33] IJSREM, “Performance and usability comparison of Python web frameworks for data science application: Django, Flask, and FastAPI,” 2023, consultado: mayo de 2025. [Online]. Available: <https://ijsrem.com/download/performance-and-usability-comparison-of-python-web-frameworks-for-data-science-application-django-flask-and-fastapi/>
- [34] Tom Christie et al., “Starlette — The little ASGI framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://starlette.dev/>
- [35] Samuel Colvin, “Pydantic — Data validation and settings management using Python type annotations,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://pydantic.dev/>
- [36] Pallets Projects, “Flask — A lightweight WSGI web application framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://flask.palletsprojects.com/>
- [37] Django Software Foundation, “Django — The web framework for perfectionists with deadlines,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://www.djangoproject.com/>
- [38] OpenAPI Initiative, “OpenAPI Specification,” 2026, especificación oficial OpenAPI. Consultado el 20 de mayo de 2026. [Online]. Available: <https://spec.openapis.org/oas/latest.html>
- [39] Neon, “Neon – serverless postgres,” 2026, consultado: 16 de mayo de 2026. [Online]. Available: <https://neon.tech/>
- [40] DataCamp, “Apache Parquet explained: A guide for data professionals,” 2025, consultado: mayo de 2025. [Online]. Available: <https://www.datacamp.com/tutorial/apache-parquet>
- [41] bckinfo.com, “Introduction to Apache Parquet: The efficient columnar storage format for big data,” 2025, consultado: mayo de 2025. [Online]. Available: <https://bckinfo.com/introduction-to-apache-parquet-the-efficient-columnar-storage-format-for-big-data/>
- [42] GIS-OPS, “Free and open source routing engines overview,” 2024, consultado: mayo de 2025. [Online]. Available: https://github.com/gis-ops/tutorials/blob/master/general/foss_routing_engines_overview.md
- [43] Google Generative AI, “Gemini 2.5 flash — especificaciones y guía de uso,” 2025, página del modelo Gemini 2.5 Flash. Consultado el 21 de mayo de 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/>

[models/gemini-2.5-flash?hl=es-419](https://cloud.google.com/models/gemini-2.5-flash?hl=es-419)

- [44] —, “Tool calling: integrar funciones externas desde modelos generativos,” 2024, documentación oficial de Google sobre tool calling. Consultado el 21 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling?hl=es-419>
- [45] Google Cloud, “Generative ai studio — documentación y guías,” 2024, documentación oficial de Google AI Studio. Consultado el 21 de mayo de 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs?hl=es-419>
- [46] —, “Cloud Build – continuous integration and delivery,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/build/docs?hl=es>
- [47] —, “Cloud Storage – object storage for developers,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/storage/docs?hl=es>
- [48] G. T. Doran, “There’s a s.m.a.r.t. way to write management’s goals and objectives,” *Management Review*, vol. 70, no. 11, p. 35, nov 1981, eBSCOhost. [Online]. Available: <https://research.ebsco.com/linkprocessor/plink?id=ad2e5b31-f1c7-323b-af72-991d4bebabf9>
- [49] Google Cloud, “Artifact Registry – store and manage build artifacts in a single location,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://cloud.google.com/artifact-registry/docs?hl=es>
- [50] Docker, Inc., “Docker – empowering app development for developers,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://www.docker.com/>
- [51] Google Cloud, “Secret Manager – secure and convenient storage for api keys, passwords, certificates, and other sensitive data,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://cloud.google.com/secret-manager/docs?hl=es>
- [52] DEUSTEK / MORElab, “Morelab - media and open research lab, universidad de deusto,” 2026, consultado el 14 de mayo de 2026. [Online]. Available: <https://morelab.deusto.es/>
- [53] I. Sommerville, *Ingeniería de software*, 9th ed. México: Pearson Educación, 2011.
- [54] Docker, Inc., “Docker Compose – Documentation,” 2026, documentación oficial. Consultado el 20 de

- mayo de 2026. [Online]. Available: <https://docs.docker.com/compose>
- [55] —, “Docker Compose — Profiles (How-to),” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://docs.docker.com/compose/how-tos/profiles/>
- [56] Google Cloud, “Cloud scheduler — cron job service,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/scheduler/docs?hl=es>
- [57] —, “Google Cloud Platform – products and services,” 2026, consultado: 16 de mayo de 2026. [Online]. Available: <https://cloud.google.com/products>
- [58] PostGIS Project, “PostGIS Documentation,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://postgis.net/documentation/>
- [59] Fastify Project, “Fastify: Fast and Low Overhead Web Framework for Node.js,” 2024, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://fastify.dev/>
- [60] —, “@fastify/jwt - Fastify JWT plugin,” 2024, repositorio/documentación. Consultado el 20 de mayo de 2026. [Online]. Available: <https://github.com/fastify/fastify-jwt>
- [61] OWASP, “Cross Site Scripting (XSS) – OWASP,” 2026, documentación/guía oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://owasp.org/www-community/attacks/xss/>
- [62] Fastify Project, “@fastify/rate-limit - Fastify rate limit plugin,” 2024, repositorio/documentación. Consultado el 20 de mayo de 2026. [Online]. Available: <https://github.com/fastify/fastify-rate-limit>